

```
In [28]: import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns

import warnings
warnings.filterwarnings(action = 'ignore')

from IPython.display import Image
```

```
In [29]: Image('167.png')
```

Out[29]:

4-2 의사 결정 나무(Decision Tree)

0. 데이터셋 소개

Titanic

[Titanic](#) 탑승객의 생존 유무를 담은 데이터셋입니다.

Data	Dictionary	Key
PassengerId	Passenger Id	Index
Survived	Survival	0 = No, 1 = Yes
Pclass	Ticket class	1 = 1st, 2 = 2nd, 3 = 3rd
Name	Name	
Sex	Sex	
Age	Age in years	
Sibsp	# of siblings / spouses aboard the Titanic	
Parch	# of parents / children aboard the Titanic	
Ticket	Ticket number	
Fare	Passenger fare	
Cabin	Cabin number	
Embarked	Port of Embarkation	C = Cherbourg, Q = Queenstown, S = Southampton

다양한 형식의 데이터를 가지고 있으며, 여러 가지 아이디어를 생각하고 시도할 만한 요소가 많게끔 기획된 데이터셋입니다.

```
In [30]: df_titanic = pd.read_csv('data/titanic.csv',
                                index_col='PassengerId')
```

```
In [31]: df_titanic.isna().sum().to_frame().T
```

```
Out[31]:
```

	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	0	0	0	0	177	0	0	0	0	687	2

전처리: Pclass와 Embarked를 가변수화 합니다.

```
In [32]: '''
1. `pd.get_dummies()` :
- `pd.get_dummies()` 함수는 데이터프레임의
범주형 변수를 원-핫 인코딩하여
더미 변수로 변환합니다.
- 이 함수는 데이터프레임을 입력으로 받고,
범주형 변수를 선택하여 더미 변수로 변환한
새로운 데이터프레임을 반환합니다.
2. `df_titanic.assign()` :
- `assign()` 메서드는 새로운 열을 추가하거나 기존 열을
수정한 후 수정된 데이터프레임을 반환합니다.
- 이 코드에서는 'Embarked' 열의 결측값을 최빈값으로
대체하고, 'Age' 열의 결측값을 평균값으로 대체합니다.
- `Embarked` 열의 결측값을 최빈값으로 대체하기 위해
`fillna()`와 `mode()` 메서드를 사용합니다.
- `Age` 열의 결측값을 평균값으로 대체하기 위해
`fillna()`와 `mean()` 메서드를 사용합니다.
3. `columns=['Pclass', 'Embarked']` :
- `get_dummies()` 함수 내에서 사용되는 매개변수로,
원-핫 인코딩할 열들을 지정합니다.
- 여기서는 'Pclass'와 'Embarked' 열이 선택되어
해당 열들에 대해 원-핫 인코딩이 수행됩니다.
4. `df_titanic.assign(Sex=lambda x: x['Sex'] == 'female')` :
- 성별 열('Sex')을 이진 변수로 변환합니다.
여성인 경우 1, 남성인 경우 0으로 인코딩됩니다.
- `assign()` 메서드를 사용하여 새로운 열을 추가합니다.
람다 함수를 사용하여 성별이 여성인지 여부를 판단합니다.
- 결과적으로 새로운 'Sex' 열은 불리언(True/False)
값을 가지며, 이를 정수형으로 변환하여
이진 변수로 만듭니다.
```

결론

이 코드는 타이타닉 데이터셋을 전처리하여 모델에 사용할 수 있는 형태로 변환합니다. 결측값을 적절히 처리하고, 범주형 변수를 원-핫 인코딩하여 이진 변수로 변환하여 데이터를 준비합니다. 이는 머신러닝 모델에 입력으로 사용하기 적합한 형태로 데이터를 가공하는 것을 목적으로 합니다.

...

...

두 번의 `assign()` 함수 호출은 각각 다른 작업을 수행합니다. 각각의 차이점을 설명해보겠습니다.

1. 첫 번째 `assign()` 함수 호출:

- 첫 번째 `assign()` 함수는 `Embarked`와 `Age` 열에 대한 전처리를 수행합니다.
- `Embarked` 열의 결측값을 최빈값으로 대체합니다.
- `Age` 열의 결측값을 평균값으로 대체합니다.
- 즉, 이 호출은 데이터프레임에 새로운 열을 추가하는 것이 아니라, 기존 열의 값을 수정합니다.

2. 두 번째 `assign()` 함수 호출:

- 두 번째 `assign()` 함수는 `Sex` 열에 대한 전처리를 수행합니다.
- `Sex` 열의 값을 불리언 형태로 변환하여 이진 변수로 만듭니다. 여성인 경우 `True`, 남성인 경우 `False`로 인코딩됩니다.
- 이 호출은 데이터프레임에 새로운 열을 추가하여 기존 열을 변형하는 것입니다.

따라서, 두 번의 `assign()` 함수 호출은 각기 다른 열을 수정하거나 변형하는 작업을 수행합니다.

첫 번째 호출은 결측값 처리와 같은 전처리 작업을 수행하고, 두 번째 호출은 범주형 변수를 이진 변수로 변환하는 작업을 수행합니다.

...

...

`mode()` 함수는 주어진 데이터에서 최빈값을 찾아줍니다. 최빈값은 데이터에서 가장 자주 등장하는 값입니다. 그리고 `iloc[0]`은 최빈값 중에서 첫 번째 값을 선택하는 것입니다.

여기서 ``mode()`` 함수가 호출되는 상황은 결측값을 처리할 때입니다. 따라서 ``x['Embarked'].mode().iloc[0]``는 'Embarked' 열에서 결측값을 대체할 값으로, 해당 열의 최빈값 중에서 첫 번째 값을 선택합니다.

예를 들어, 'Embarked' 열이 ['S', 'C', 'S', 'Q', 'S']와 같이 주어졌을 때, 'S'가 가장 자주 등장하므로 ``mode()`` 함수는 ['S']를 반환합니다. 따라서 ``iloc[0]``을 사용하여 이 중 첫 번째 값을 선택하면 'S'가 선택됩니다. 이 값은 결측값을 대체하는데 사용됩니다.

일반적으로 데이터프레임이나 시리즈에 여러 개의 최빈값이 있다면, ``mode()`` 함수는 모든 최빈값을 반환합니다. 그러나 이 경우에는 최빈값 중 첫 번째 값을 선택하여 결측값을 대체하려는 것이므로, ``.iloc[0]``을 추가하여 최빈값 중 첫 번째 값을 선택합니다.

이렇게 하면 코드가 조금 더 명확해집니다. 그러나 최빈값이 하나밖에 없는 경우에는 ``.iloc[0]``을 사용하지 않아도 됩니다. 일반적으로 ``.iloc[0]``을 사용하여 명시적으로 첫 번째 값을 선택하는 것이 좋습니다.

```
'''
# Embarked의 결측치는 최빈값으로,
# Age의 결측치는 평균으로 대체합니다.
# 범주형 변수는 가변수화하여 수치형 변수화 동일하게 취급합니다.
# Sex은 female, male 두 개의 범주이므로 이진형으로 바꿉니다.
df_titanic = pd.get_dummies(
    df_titanic.assign(
        Embarked = lambda x: x['Embarked']\
            .fillna(x['Embarked'].mode().iloc[0]),
        Age = lambda x: x['Age'].fillna(x['Age'].mean()),
    ), columns=['Pclass', 'Embarked']
).assign(
    Sex = lambda x: x['Sex'] == 'female'
)

df_titanic.head()
```

Out[32]:

	Survived	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Pclass_1	Pclass_2	Pclass_3	Embarked_C	Embarked
PassengerId														
1	0	Braund, Mr. Owen Harris	False	22.0	1	0	A/5 21171	7.2500	NaN	False	False	True	False	
2	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	True	38.0	1	0	PC 17599	71.2833	C85	True	False	False	True	
3	1	Heikkinen, Miss. Laina	True	26.0	0	0	STON/O2. 3101282	7.9250	NaN	False	False	True	False	
4	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	True	35.0	1	0	113803	53.1000	C123	True	False	False	False	
5	0	Allen, Mr. William Henry	False	35.0	0	0	373450	8.0500	NaN	False	False	True	False	

In [33]:

```
'''
코드는 타이타닉 데이터셋에서 수치형 특성과
범주형 특성을 구분하여 리스트로 정의하는
작업을 수행합니다.

1. `X_num = ['Age', 'SibSp', 'Parch', 'Fare']`:
- `X_num`은 수치형 특성들을 담은 리스트입니다.
- 여기서 'Age', 'SibSp', 'Parch', 'Fare'는
타이타닉 데이터셋에서 수치형으로
```

처리되는 특성들입니다.

2. `X_dum = [i for i in df_titanic.columns
 if i.startswith('Pclass') or
 i.startswith('Sex') or
 i.startswith('Embarked')]`:
 - `X_dum`은 범주형 특성들을 담은 리스트입니다.
 - 여기서는 데이터프레임의 열 이름이
 `'Pclass'`, 'Sex'`, 'Embarked'`로
 시작하는 열들을 선택합니다.`
3. `X_all = X_num + X_dum``:
 - `X_all`은 수치형 특성과 범주형 특성을
 모두 포함하는 리스트입니다.`
 - `X_num`과 X_dum`의 모든 요소들을 합친
 결과가 됩니다.`
 - 따라서 `X_all` 리스트에는 수치형 특성과
 범주형 특성이 모두 포함됩니다.`

결론

코드는 타이타닉 데이터셋에서 수치형 특성과
범주형 특성을 구분하여 각각의 리스트에 저장하고,
이를 합쳐서 모든 특성을 포함하는 리스트를 생성합니다.
이렇게 하면 데이터를 분석하거나 모델에 입력으로
사용할 때 각각의 특성 유형을 쉽게 구분할 수 있습니다.
'''

머신러닝에 사용할 변수들을 정리합니다.

```
X_num = ['Age', 'SibSp', 'Parch', 'Fare']
X_dum = [i for i in df_titanic.columns \
          if i.startswith('Pclass') or \
          i.startswith('Sex') or \
          i.startswith('Embarked')]
X_all = X_num + X_dum
X_all
```

```
Out[33]: ['Age',
          'SibSp',
          'Parch',
          'Fare',
          'Sex',
          'Pclass_1',
          'Pclass_2',
          'Pclass_3',
          'Embarked_C',
          'Embarked_Q',
          'Embarked_S']
```

```
In [34]: Image('168.png')
```

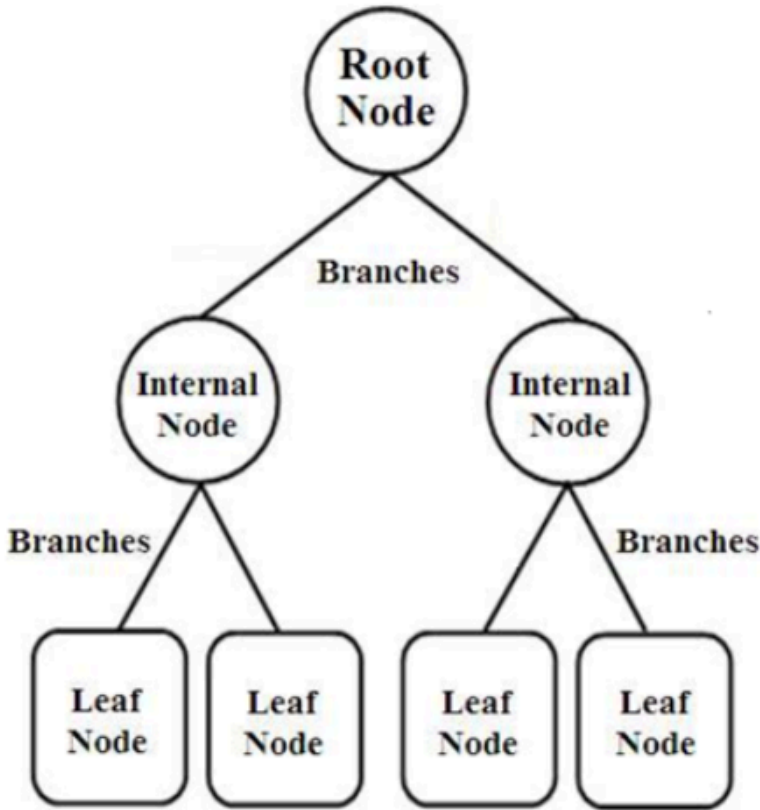
Out[34]: **1. 종류 및 특징**

모델명	특징
ID3 (Iterative Dichotomiser 3)	1. 분류에 사용됩니다. 2. 범주형 입력 데이터만 지원합니다.
C4.5	1. 연속형 입력 데이터도 지원합니다. 2. 의사 결정 나무의 구조를 만든 후에 가지치기를 지원합니다.
CART (Classification and Regression Trees)	1. 분류와 회귀 에 사용됩니다.
CHAID (Chi-squared Automatic Interaction Detector)	1. 범주형 입력 데이터만 지원합니다. 2. 두 개 이상의 가지를 지닙니다. 3. 분류에서는 Chi2 검정통계량을 회귀에서는 ANOVA 검정통계량을 사용합니다.

```
In [35]: Image('169.png')
```


Out[35]: **2. 구성 요소**

요소명	설명
내부 노드(Internal Node)	분기 조건과 결정 경로를 나타내는 가지(Branch)로 구성된 노드입니다. 이 중 최상위 노드를 루트 노드(Root Node)라고 합니다.
가지(Branch)	분기 조건에 해당하는 후속 노드와의 연결 관계를 나타내는 요소입니다.
리프 노드(Leaf Node)	가장 하위에 있는 노드로 더 이상 분기가 이루어지지 않고, 최종 예측 결과를 지니고 있는 노드 입니다.



In [36]: `Image('170.png')`

Out[36]: **3.의사 결정 나무 학습**

분기 조건의 선정

- 내부 노드(Internal Node)를 구성하는 분기 조건을 도출합니다.

분기 조건의 표현

- 범주형일 경우 '변수명 = 범주', 연속형일 경우 '변수명 < 분리점'
- scikit-learn의 Decision Tree를 기준으로 설명합니다.

scikit-learn의 의사 결정 나무는 범주형은 가변수화를 하여 연속형, 범주형 동일하게 취급하고 '변수명 <= 분리점'으로 조건을 나타냅니다.

불순도

노드에 부여된 데이터의 대상 변수의 클래스가 혼재하는 정도를 나타냅니다.

분류

1. 엔트로피(Entropy)

$H(t) = - \sum_{i=1}^c p_i \log_2 p_i$, t: 노드, c: 클래스의 수, p_i : i 클래스의 비율(출현 확률)

p_i 가 모두 동일할 경우 즉, 모든 확률이 $1/c$ 에 가까울 수록 엔트로피는 커집니다.

2. 지니 불순도(Gini impurity)

$G(t) = \sum_{i=1}^c p_i(1 - p_i) = \sum_{i=1}^c (p_i - p_i^2) = 1 - \sum_{i=1}^c p_i^2$

In [37]: `'''
코드는 이진 분류의 엔트로피(Entropy)와
지니 불순도(Gini impurity)를 계산하고`

시각화하는 작업을 수행합니다.

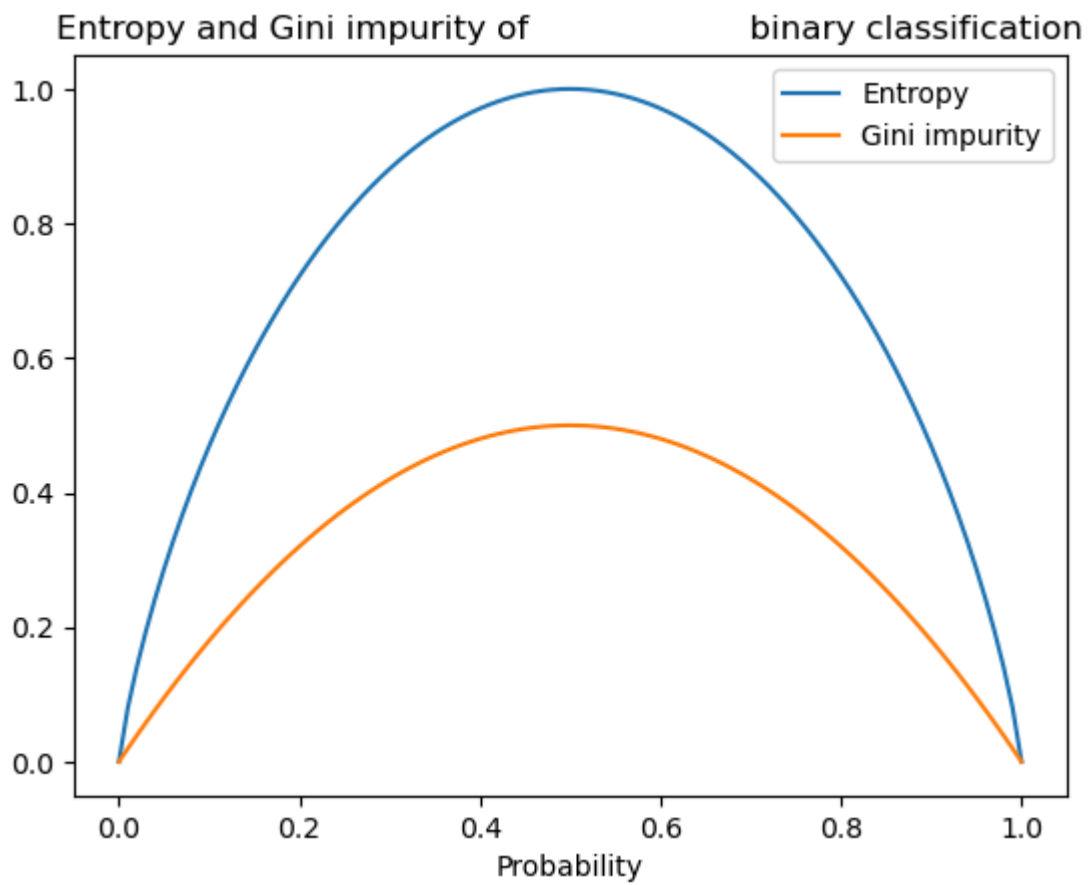
1. ``x = np.linspace(0, 1, 101)``:
 - ``x``는 0부터 1까지 101개의 균등하게 나누어진 값을 갖는 배열입니다. 이 배열은 확률 값을 나타냅니다.
 - ``np.linspace(0, 1, 101)``는 0과 1 사이를 100개의 구간으로 나누어 101개의 값을 생성합니다.
2. ``x2 = x[1:-1]``:
 - ``x2``는 ``x`` 배열에서 첫 번째와 마지막 값을 제외한 중간 값을 가지는 배열입니다.
 - 이는 ``x`` 배열의 2번째 요소부터 마지막에서 두 번째 요소까지를 선택합니다.
3. ``e = np.hstack([[0], -(x2 * np.log2(x2) + (1 - x2) * np.log2(1 - x2))], [0])``:
 - 이 줄은 엔트로피를 계산하는 작업을 수행합니다.
 - ``x2 * np.log2(x2) + (1 - x2) * np.log2(1 - x2)``는 이진 분류에서의 엔트로피 식입니다.
 - ``-`` 부호는 엔트로피 계산의 특성상 값이 음수가 되지 않도록 하기 위해 사용됩니다.
 - ``np.hstack([[0], ... , [0]])``는 엔트로피 배열의 앞뒤에 0을 추가합니다. 이는 확률이 0이나 1일 때 엔트로피가 0이 되기 때문입니다.
4. ``plt.plot(x, e, label='Entropy')``:
 - ``x``와 ``e`` 값을 사용하여 엔트로피를 그래프로 그립니다.
 - ``label='Entropy'``는 그래프의 범례에서 이 선을 'Entropy'로 표시합니다.
5. ``plt.plot(x, 1 - (x ** 2 + (1 - x) ** 2), label='Gini impurity')``:
 - 이 줄은 지니 불순도를 계산하고 그래프로 그립니다.
 - ``1 - (x ** 2 + (1 - x) ** 2)``는 이진 분류에서의 지니 불순도 식입니다.
 - ``label='Gini impurity'``는 그래프의 범례에서 이 선을 'Gini impurity'로 표시합니다.
6. ``plt.gca().set_xlabel('Probability')``:
 - x축 레이블을 'Probability'로 설정합니다.
7. ``plt.gca().set_title('Entropy and Gini impurity of binary classification')``:
 - 그래프의 제목을 'Entropy and Gini impurity of binary classification'로 설정합니다.
8. ``plt.legend()``:
 - 범례를 추가하여 그래프에 표시된 선들의 의미를 나타냅니다.

결론

이 코드는 이진 분류에서 사용되는 두 가지 주요 불확실성 측정 방법인 엔트로피와 지니 불순도의 값들을 확률에 따라 계산하고, 이를 시각화하여 비교합니다. 엔트로피는 확률이 0이나 1일 때 0이 되고, 확률이 0.5일 때 최대값을 가지며, 지니 불순도는 확률이 0이나 1일 때 0이 되고, 확률이 0.5일 때 최대값을 가지는 것을 시각적으로 확인할 수 있습니다.

```
'''
x = np.linspace(0, 1, 101)
x2 = x[1:-1]
e = np.hstack([[0], -(x2 * np.log2(x2) + \
                    (1 - x2) * np.log2(1 - x2)), [0]])
plt.plot(x, e, label='Entropy')
plt.plot(x, 1 - (x ** 2 + (1-x) ** 2),
         label='Gini impurity')
plt.gca().set_xlabel('Probability')
plt.gca().set_title('Entropy and Gini impurity of \
                    binary classification')

plt.legend()
plt.show()
'''
```



```
In [38]: Image('171.png')
```

Out[38]: 회귀

- 평균제곱오차(Mean Squared Error)

$$MSE(t) = \frac{1}{n} \sum_{i=1}^c \sum_{j=1}^{n_i} (y_{ij} - \bar{y}_i)^2, c: \text{클래스의 수}, n_i: i \text{ 클래스의 데이터수}$$

- 평균절대오차(Mean Absolute Error)

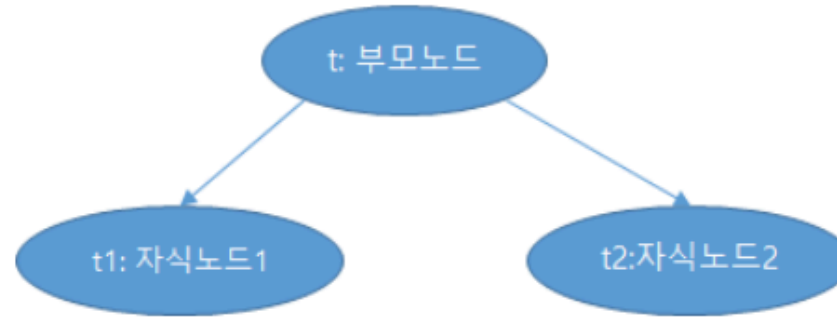
$$MAE(t) = \frac{1}{n} \sum_{i=1}^c \sum_{j=1}^{n_i} |y_{ij} - \text{median}(y)_i|, c: \text{클래스의 수}, n_i: i \text{ 클래스의 데이터수}$$

In [39]: Image('172.png')

Out[39]: 불순도 감소량

데이터의 분기에 의해 감소된 불순도

▷ 부모 노드의 불순도에서 자식노드들의 불순도(데이터수에 대한 불순도의 가중합)를 뺀 값입니다.



$$IG(t)(t\text{노드에 의해 감소된 불순도}) = imp(t) - \left(\frac{N(t_1)}{N(t)} imp(t_1) + \frac{N(t_2)}{N(t)} imp(t_2) \right)$$

$imp(t)$: t 노드의 불순도, $N(t)$: t 노드의 데이터수

특히, 엔트로피(Entropy)를 불순도로 사용했을 때 불순도 획득을 **정보 획득(Information Gain)**이라고 합니다.

Gini Index vs Entropy의 차이

Gini Impurity가 Entropy보다 스케일이 작기 때문에 자식노드의 데이터수의 가중치가 불순도 차감을 계산할 때 더 큰 영향을 주게 됩니다.

따라서 Gini Impurity로 불순도 차감을 계산할 때, 상대적으로 데이터의 수가 많은 노드들 위주로 분기가 이루어집니다.

[Ex.1]

df_titanic을 대상 변수 Survived에 대한

가변수 Parch 분기했을 때, Entropy로 불순도 차감을 계산합니다.

Parch 뿐만 아니라 일반적인 연속형 변수에서도

동작 가능하게 구현합니다.

```
In [40]: def entropy(x):
        """
        엔트로피를 구합니다.
        Parameters:
            x: array-like
        Returns:
            float: 엔트로피
        """
        return -np.sum(x * np.log2(np.where(x > 0, x, 1)))

def gini(x):
    """
    지니 불순도를 구합니다.
    Parameters:
        x: array-like
    Returns:
        float: 지니불순도
    """
    return 1 - np.sum(x ** 2)
```

```
In [41]: df_titanic['Survived'].value_counts(normalize=True)
```

```
Out[41]: Survived
0      0.616162
1      0.383838
Name: proportion, dtype: float64
```

```
In [42]: # df_titanic에서 Survived의 불순도를 구합니다.
imp_func = entropy # entropy를 불순도로 사용합니다.
#imp_func = gini
imp_t = imp_func(df_titanic['Survived'])
```



```
.value_counts(normalize=True))
imp_t
```

Out[42]: 0.9607079018756469

In [43]: Image('173.png')

Out[43]: 분기점 별 좌측 노드와 우측 노드의 엔트로피 구하기

1. 변수 x 의 모든 분기 지점을 구합니다.

등장하는 숫자 집합을 구합니다. $a_1, a_2, \dots, a_n$ 모든 i 에 대해서 $a_i < a_{i+1}$ 입니다.

모든 i 에 대하여, a_i 와 a_{i+1} 사이의 중간값으로 분기점을 만듭니다.: $b_i = \frac{a_i + a_{i+1}}{2}$

2. 분기점 별로 좌측 노드의 대상 클래스 빈도수를 구합니다.

3. 분기점 별로 좌측 노드의 클래스 빈도수를 이용하여, 좌/우측 노드의 불순도를 구합니다.

```
In [44]: '''
코드는 타이타닉 데이터셋에서 'Parch' 변수와
'Survived' 변수 간의 관계를 분석하는 과정을 수행합니다.

1. `u = pd.Series(df_titanic['Parch'].unique())
   .sort_values()` :
   - `df_titanic['Parch'].unique()`는 'Parch' 열의
     고유한 값을 반환합니다.
   - `pd.Series()`는 이 고유한 값들을 시리즈로 변환합니다.
   - `sort_values()`는 이 시리즈를 오름차순으로 정렬합니다.
   - 결과적으로, `u`는 'Parch' 열의 고유한 값들을
     오름차순으로 정렬한 시리즈입니다.
2. `b = np.array([-np.inf] + ((u + u.shift(1)) / 2)\
   .dropna().tolist() + [np.inf])` :
   - `u.shift(1)`은 `u` 시리즈의 각 값을 한 칸씩 아래로
     이동시킵니다. 첫 번째 값은 NaN이 됩니다.
   - `(u + u.shift(1)) / 2`는 `u` 시리즈와 한 칸씩 이동한
     `u` 시리즈를 더한 후 2로 나누어 중간 값을 계산합니다.
```

```

- `dropna()`는 NaN 값을 제거합니다.
- `tolist()`는 이 결과를 리스트로 변환합니다.
- `np.array([-np.inf] + ... + [np.inf])`는 이 리스트
  앞뒤에 각각 -무한대와 무한대를 추가하여
  numpy 배열로 변환합니다.
- 결과적으로, `b`는 'Parch' 값들을 구간으로 나누기 위한
  경계 값을 포함하는 배열입니다.
3. `n = len(df_titanic)` :
  - `n`은 타이타닉 데이터셋의 전체 행 수를 저장합니다.
  - `len(df_titanic)`은 데이터프레임의 행 수를 반환합니다.
4. `df_cnt = pd.crosstab(index=pd.cut(df_titanic['Parch'],
  bins=b, right=True), columns=df_titanic['Survived'])
  .cumsum().rename(index=lambda x: "Parch <= {}".format(x.right))` :
  - `pd.cut(df_titanic['Parch'], bins=b, right=True)`는
    'Parch' 값을 `b` 배열의 경계 값을 기준으로
    구간으로 나눕니다. `right=True`는 각 구간의
    오른쪽 경계 값을 포함한다는 의미입니다.
  - `pd.crosstab(index=..., columns=df_titanic['Survived'])`는
    각 구간에 대해 'Survived' 변수의 교차 테이블을 생성합니다.
  - `cumsum()`는 각 구간별로 누적 합계를 계산합니다.
  - `rename(index=lambda x: "Parch <= {}".format(x.right))`는
    인덱스를 새로운 구간 레이블로 변경합니다.
    여기서 `x.right`는 구간의 오른쪽 경계 값을 나타냅니다.
  - 결과적으로, `df_cnt`는 'Parch' 값이 특정 구간 이하인 경우에
    대해 'Survived' 변수의 누적 합계를 나타내는 데이터프레임입니다.

```

결론

이 코드는 'Parch' 변수의 고유 값을 기준으로 구간을 나누고,
 각 구간에 대해 생존자 수와 사망자 수의 누적 합계를 계산하여
 데이터프레임으로 나타냅니다. 각 구간은 "Parch <= 경계값" 형식으로
 레이블링 되어 있으며, 이를 통해 'Parch' 값과
 생존 여부 간의 관계를 분석할 수 있습니다.

```
'''
```

```

# Parch에 등장하는 숫자에서
# 고유값을 구하고 이를 정렬합니다.: a1 ~ an 구하기
u = pd.Series(df_titanic['Parch'].unique()).sort_values()
# Parch에 등장하는 숫자에서 만들 수 있는 분기점을 구합니다.
# 효율적이고 간편하게 분기점별 좌측 노드의

```

```

# 클래스 빈도수를 구하기 위해
# 모든 bi 와 bi+1 분기 구간에 등장하는
# 클래스 빈도수를 구하고
# 이를 분기 구간별로 정렬된 상태에서
# 빈도수의 누계를 구하여 bi+1를 기준으로
# 좌측 노드의 클래스의 빈도와 우측 노드의
# 클래스의 빈도를 구합니다.
b = np.array([-np.inf] + ((u + u.shift(1)) / 2)\
              .dropna().tolist() + [np.inf])
n = len(df_titanic)
df_cnt = pd.crosstab(
    index = pd.cut(df_titanic['Parch'], bins=b, right=True),
    columns = df_titanic['Survived']
).cumsum().rename(index=lambda x: "Parch <= {}".format(x.right))
df_cnt

```

Out[44]:

	Survived	0	1
Parch			
Parch <= 0.5	445	233	
Parch <= 1.5	498	298	
Parch <= 2.5	538	338	
Parch <= 3.5	540	341	
Parch <= 4.5	544	341	
Parch <= 5.5	548	342	
Parch <= inf	549	342	

[참고]

연속형 변수의 최적의 분기점을 찾기 위해서는

최악의 경우에는 데이터의 수만큼 선형 탐색을 해야 합니다.

백분위 기반으로 연속형 변수를 구간화하여

분기점 찾는 속도를 효율화하는 방법이 널리 쓰입니다.

<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.HistGradientBoostingRegressor.html>

In [45]:

```
'''
코드는 `df_cnt` 데이터프레임에 대해 사용자 정의 함수
`imp_func`를 적용하여 각 행에 대한 불확실성 측정값과
가중치를 계산합니다.

1. `df_cnt.apply(..., axis=1)`:
  - `apply` 함수는 데이터프레임의 각 행 또는 열에
    함수를 적용합니다.
  - `axis=1`은 함수를 각 행에 적용한다는 것을 의미합니다.
    즉, 데이터프레임의 각 행이 함수에 전달됩니다.
2. `lambda x: pd.Series([...], index=['imp', 'weight'])`:
  - `lambda x`는 각 행을 입력으로 받는 람다 함수입니다.
  - 이 람다 함수는 두 가지 값을 가지는 시리즈를
    반환합니다: 불확실성 측정값(`imp`)과 가중치(`weight`).
3. `imp_func(x / x.sum())`:
  - `x`는 `df_cnt` 데이터프레임의 각 행입니다.
  - `x.sum()`은 해당 행의 합계입니다.
    이는 그 구간에 속하는 데이터의 총 개수를 의미합니다.
  - `x / x.sum()`은 행의 각 값을 그 행의 합계로 나누어
    확률 분포를 만듭니다.
  - `imp_func`는 이 확률 분포에 대해 불확실성 측정값을
    계산하는 함수입니다. 여기서는 `entropy` 함수가
    `imp_func`로 설정되어 있습니다.
4. `pd.Series([imp_func(x / x.sum()), x.sum() / n],
              index=['imp', 'weight'])`:
  - `pd.Series`는 새로운 시리즈 객체를 생성합니다.
  - `[imp_func(x / x.sum()), x.sum() / n]`는 시리즈의 값들입니다.
    첫 번째 값은 불확실성 측정값(`imp`),
    두 번째 값은 가중치(`weight`)입니다.
  - `index=['imp', 'weight']`는 시리즈의 인덱스를 설정합니다.
  - `x.sum() / n`은 각 행의 합계를 데이터프레임의 총 행 수
    `n`으로 나누어 해당 구간의 가중치를 계산합니다.
'''
```

결론

이 코드는 각 구간에 대해 불확실성 측정값 (여기서는 엔트로피)을 계산하고, 해당 구간의 가중치를 계산하여 새로운 시리즈로 반환합니다. 이 작업은 데이터프레임의 각 행에 대해 수행되며, 결과는 두 개의 열('imp'와 'weight')을 가지는 새로운 데이터프레임이 됩니다. 이를 통해 각 구간의 불확실성과 그 구간의 중요도를 파악할 수 있습니다.

전체 동작 과정 예시

1. `df\_cnt` 데이터프레임이 다음과 같이 생겼다고 가정합니다:

```
Parch <= 0    5    10
Parch <= 1   15    20
Parch <= 2   25    30
```

2. 각 행에 대해 `apply` 함수가 작동합니다.

```
- 첫 번째 행: `[5, 10]`
  - `x / x.sum()`: `[5/15, 10/15]` = `[1/3, 2/3]`
  - `imp_func([1/3, 2/3])`: 엔트로피 값 계산
  - `x.sum() / n`: `15 / (전체 데이터 수)`
  - 결과 시리즈: `[엔트로피 값, 가중치]`
- 두 번째 행: `[15, 20]`
  - `x / x.sum()`: `[15/35, 20/35]`
  - `imp_func([15/35, 20/35])`: 엔트로피 값 계산
  - `x.sum() / n`: `35 / (전체 데이터 수)`
  - 결과 시리즈: `[엔트로피 값, 가중치]`
```

3. 최종 결과는 각 행에 대한 엔트로피와 가중치를 포함하는 데이터프레임입니다:

```
      imp    weight
Parch <= 0  엔트로피 값  가중치
Parch <= 1  엔트로피 값  가중치
Parch <= 2  엔트로피 값  가중치
...
```

좌측 노드의 불순도와 가중치를 구합니다.

```
df_cnt.apply(
    lambda x: pd.Series([imp_func(x / x.sum()), x.sum() / n],
```

```
index=['imp', 'weight']), axis=1
)
```

Out[45]:

	imp	weight
Parch		
Parch <= 0.5	0.928276	0.760943
Parch <= 1.5	0.953970	0.893378
Parch <= 2.5	0.962066	0.983165
Parch <= 3.5	0.962876	0.988777
Parch <= 4.5	0.961707	0.993266
Parch <= 5.5	0.961002	0.998878
Parch <= inf	0.960708	1.000000

```
In [46]: df_titanic['Survived'].value_counts().values
```

```
Out[46]: array([549, 342], dtype=int64)
```

In [47]:

```
'''
코드는 타이타닉 데이터셋에서 'Survived' 변수와
'Parch' 변수를 사용하여 생존자와 사망자의 분포를 분석하고,
이를 통해 불확실성 측정값(예: 엔트로피)과
가중치를 계산하는 작업을 수행합니다.

1. `df_titanic['Survived'].value_counts().values`:
  - `df_titanic['Survived'].value_counts()`는
    'Survived' 열의 값의 빈도를 계산합니다.
  - `value_counts().values`는 그 결과를 배열로 반환합니다.
    예를 들어, `[사망자 수, 생존자 수]`와
    같은 배열을 얻을 수 있습니다.
2. `df_titanic['Survived'].value_counts().values - df_cnt`:
  - `df_cnt`는 이전에 정의된 데이터프레임으로,
    'Parch' 값에 따른 생존자 수와 사망자 수의
    누적 합계를 포함합니다.
'''
```

```
- `df_titanic['Survived'].value_counts().values`는
  전체 데이터에 대한 생존자 수와 사망자 수를
  나타내는 배열입니다.
- `df_titanic['Survived'].value_counts().values - df_cnt`는
  각 구간에 대해 전체 생존자 수와 사망자 수에서
  해당 구간의 누적 생존자 수와 사망자 수를 뺀 값을 계산합니다.
  이는 각 구간 외부의 생존자 수와 사망자 수를 나타냅니다.
3. `apply(lambda x: pd.Series([imp_func(x / x.sum()),
  x.sum() / n], index=['imp', 'weight']), axis=1)`:
- `apply` 함수는 데이터프레임의 각 행에 함수를 적용합니다.
  여기서는 각 구간 외부의 생존자 수와 사망자 수에
  대해 작업을 수행합니다.
- `lambda x`는 각 행을 입력으로 받는 람다 함수입니다.
- `x / x.sum()`은 행의 각 값을 그 행의 합계로 나누어
  확률 분포를 만듭니다.
- `imp_func(x / x.sum())`은 이 확률 분포에 대해 불확실성
  측정값을 계산하는 함수입니다. 여기서는 `entropy` 함수가
  `imp_func`로 설정되어 있습니다.
- `x.sum() / n`은 각 행의 합계를 데이터프레임의
  총 행 수 `n`으로 나누어 해당 구간의 가중치를 계산합니다.
- `pd.Series([imp_func(x / x.sum()), x.sum() / n],
  index=['imp', 'weight'])`는 불확실성 측정값(`imp`)과
  가중치(`weight`)를 가지는 시리즈를 생성합니다.
```

결론

이 코드는 타이타닉 데이터셋에서 'Parch' 값에 따른 생존자 수와 사망자 수의 분포를 분석하고, 각 구간 외부의 생존자 수와 사망자 수에 대해 불확실성 측정값(엔트로피)과 가중치를 계산합니다. 결과적으로, 각 구간 외부의 불확실성과 중요도를 시각적으로 비교할 수 있습니다.

전체 동작 과정 예시

1. `df\_titanic['Survived'].value\_counts().values`가
`[사망자 수, 생존자 수] = [424, 290]`라고 가정합니다.
2. `df\_cnt` 데이터프레임이 다음과 같이 생겼다고 가정합니다:

```
      0    1
Parch <= 0  300  150
Parch <= 1  350  200
Parch <= 2  400  250
```

3. `(df_titanic['Survived'].value_counts().values

- df_cnt)` 계산은 각 구간에 대해 다음과 같습니다:

```

      0      1
Parch <= 0 124 140 # [424 - 300, 290 - 150]
Parch <= 1  74  90 # [424 - 350, 290 - 200]
Parch <= 2  24  40 # [424 - 400, 290 - 250]

```

4. `apply` 함수는 각 행에 대해 불확실성 측정값과 가중치를 계산합니다.

```

- 첫 번째 행: `[124, 140]`
  - `x / x.sum()`: `[124/264, 140/264]`
  - `imp_func([124/264, 140/264])`: 엔트로피 값 계산
  - `x.sum() / n`: `264 / 전체 데이터 수`
  - 결과 시리즈: `[엔트로피 값, 가중치]`
- 두 번째 행: `[74, 90]`
  - `x / x.sum()`: `[74/164, 90/164]`
  - `imp_func([74/164, 90/164])`: 엔트로피 값 계산
  - `x.sum() / n`: `164 / 전체 데이터 수`
  - 결과 시리즈: `[엔트로피 값, 가중치]`
- 세 번째 행: `[24, 40]`
  - `x / x.sum()`: `[24/64, 40/64]`
  - `imp_func([24/64, 40/64])`: 엔트로피 값 계산
  - `x.sum() / n`: `64 / 전체 데이터 수`
  - 결과 시리즈: `[엔트로피 값, 가중치]`

```

5. 최종 결과는 각 구간 외부에 대한 엔트로피와 가중치를 포함하는 데이터프레임입니다:

```

      imp      weight
Parch <= 0  엔트로피 값  가중치
Parch <= 1  엔트로피 값  가중치
Parch <= 2  엔트로피 값  가중치
...

```

```

# 우측 노드의 빈도수
#      = 전체 클래스 빈도수 - 좌측 노드 빈도수
# 우측 노드의 불순도와 가중치를 구합니다.
(df_titanic['Survived'].value_counts().values - df_cnt).apply(
    lambda x: pd.Series([imp_func(x / x.sum()), x.sum() / n],
                        index=['imp', 'weight']), axis=1
)

```


Out[47]:

	imp	weight
Parch		
Parch <= 0.5	0.999602	0.239057
Parch <= 1.5	0.996080	0.106622
Parch <= 2.5	0.836641	0.016835
Parch <= 3.5	0.468996	0.011223
Parch <= 4.5	0.650022	0.006734
Parch <= 5.5	-0.000000	0.001122
Parch <= inf	-0.000000	0.000000

In [48]:

```
'''
코드는 타이타닉 데이터셋에서 'Parch' 변수에 따라
생존자 수와 사망자 수를 분석하여 각 구간에 대해
가중치가 적용된 불확실성 측정값을 계산합니다.
이를 통해 각 구간에 대한 전체 불확실성을 계산합니다.
코드는 다음 두 부분으로 나뉩니다:

1. 구간 내부 불확실성 계산
2. 구간 외부 불확실성 계산

1. 구간 내부 불확실성 계산:
- `df_cnt`는 'Parch' 값에 따른 생존자 수와 사망자 수의
  누적 합계를 포함하는 데이터프레임입니다.
- `apply(lambda x: imp_func(x / x.sum()) * x.sum() / n,
  axis=1)`는 각 행에 대해 불확실성 측정값을
  계산하고, 이를 해당 행의 가중치로 곱합니다.
- `x`는 `df_cnt`의 각 행입니다.
- `x.sum()`은 행의 총합으로, 해당 구간의
  총 샘플 수를 의미합니다.
- `x / x.sum()`은 각 값을 총합으로 나눈 것으로,
  확률 분포를 만듭니다.
- `imp_func(x / x.sum())`는 이 확률 분포에 대해
  불확실성(예: 엔트로피)을 계산합니다.
'''
```

- ``imp_func(x / x.sum()) * x.sum() / n``은 계산된 불확실성에 해당 구간의 가중치를 곱한 것입니다.
- 결과적으로, 이 부분은 각 구간의 가중치가 적용된 불확실성을 계산합니다.

2. 구간 외부 불확실성 계산:

- ``df_titanic['Survived'].value_counts()``는 생존자와 사망자의 총수를 계산합니다.
- ``df_titanic['Survived'].value_counts() - df_cnt``는 각 구간 외부의 생존자와 사망자 수를 계산합니다.
- ``apply(lambda x: imp_func(x / x.sum()) * x.sum() / n, axis=1)``는 구간 외부의 불확실성을 계산하고, 이를 해당 구간의 가중치로 곱합니다.
- ``x``는 ``df_titanic['Survived'].value_counts() - df_cnt``의 각 행입니다.
- ``x.sum()``은 행의 총합으로, 해당 구간 외부의 총 샘플 수를 의미합니다.
- ``x / x.sum()``은 각 값을 총합으로 나눈 것으로, 확률 분포를 만듭니다.
- ``imp_func(x / x.sum())``는 이 확률 분포에 대해 불확실성(예: 엔트로피)을 계산합니다.
- ``imp_func(x / x.sum()) * x.sum() / n``은 계산된 불확실성에 해당 구간의 가중치를 곱한 것입니다.
- 결과적으로, 이 부분은 각 구간 외부의 가중치가 적용된 불확실성을 계산합니다.

3. 두 계산의 합:

- 첫 번째 부분에서 각 구간 내부의 가중치가 적용된 불확실성 값을 계산합니다.
- 두 번째 부분에서 각 구간 외부의 가중치가 적용된 불확실성 값을 계산합니다.
- 이 두 결과를 더하여 각 구간의 전체 불확실성을 계산합니다.

결론

이 코드는 각 구간에 대해 가중치가 적용된 불확실성 값을 계산하여, 구간 내부와 외부의 불확실성 값을 합산합니다. 이를 통해 각 구간의 전체 불확실성을 나타내는 값을 얻을 수 있습니다. 최종 결과는 ``df_imp``라는 데이터프레임으로, 각 구간에 대한 불확실성 값을 포함합니다.

```
'''
```

```
# 좌측 노드와 우측 노드의 불순도를 가중합을 구합니다.
df_imp = df_cnt.apply(
    lambda x: imp_func(x / x.sum()) * x.sum() / n, axis=1
) + \
(df_titanic['Survived'].value_counts() - df_cnt).apply(
    lambda x: imp_func(x / x.sum()) * x.sum() / n, axis=1
)
df_imp
```

```
Out[48]: Parch
Parch <= 0.5    0.945327
Parch <= 1.5    0.958460
Parch <= 2.5    0.959954
Parch <= 3.5    0.957333
Parch <= 4.5    0.959608
Parch <= 5.5    0.959923
Parch <= inf    0.960708
dtype: float64
```

```
In [49]: '''
코드는 `imp_t`에서 `df_imp`를 빼는 연산을 수행합니다.
이 연산의 목적은 두 데이터프레임의 차이를 계산하는 것입니다.
이를 통해 두 데이터프레임 간의 차이를 비교할 수 있습니다.

1. `imp_t`:
- `imp_t`는 불확실성 측정값을 포함하는 데이터프레임입니다.
  일반적으로, 이는 전체 데이터에 대한 불확실성 값을 나타냅니다.
- `imp_t`는 이전 코드에서 계산되었을 가능성이 큼니다.
  예를 들어, 전체 데이터셋의 엔트로피나 지니 불순도를
  나타낼 수 있습니다.

2. `df_imp`:
- `df_imp`는 각 구간에 대해 가중치가 적용된 불확실성 값을
  포함하는 데이터프레임입니다.
- `df_imp`는 구간 내부와 외부의 불확실성을 합산한 결과입니다.
  이는 특정 조건이나 구간에 대한 불확실성 값을 나타냅니다.

### 의미
- `imp_t - df_imp`는 각 구간에 대해 전체 불확실성 값과
  가중치가 적용된 불확실성 값의 차이를 나타냅니다.
- 이 차이는 각 구간에 대한 추가적인 불확실성을 나타낼 수 있습니다.
'''
```

예를 들어, ``imp_t``는 전체 데이터셋의 엔트로피이고,
``df_imp``는 특정 구간에 대한 가중치가 적용된 엔트로피 값이라면,
``imp_t - df_imp``는 각 구간이 추가하는 불확실성을 나타냅니다.

이 연산을 통해 각 구간의 중요성을 평가하거나,
 특정 구간의 불확실성 기여도를 파악할 수 있습니다.
 이는 데이터 분석 및 모델 평가에 유용할 수 있습니다.
 ...

불순도 감소를 계산합니다. `imp_t` 분기 이전의 불순도
`imp_t - df_imp`

```
Out[49]: Parch
Parch <= 0.5    0.015381
Parch <= 1.5    0.002248
Parch <= 2.5    0.000754
Parch <= 3.5    0.003375
Parch <= 4.5    0.001100
Parch <= 5.5    0.000785
Parch <= inf    0.000000
dtype: float64
```

[Ex.2]

df_titanic 에서 gini 불순도를 사용하여

X_all 변수 리스트 각각의 Survived 대한

불순도 차감을 구해봅니다.

그리고 sklearn.tree.DecisionTreeClassifier 를

X_all을 입력 변수로 대상 변수 Survived를

최대 깊이(max_depth)를 1로 설정하여 학습하고,

나무를 출력하여 직접 계산한 결과와 비교해봅니다.

```
In [50]: Image('174.png')
```

Out[50]: 정보 이득(Information Gain, IG)은 의사결정트리(Decision Tree)와 같은 기계 학습 알고리즘에서 노드를 분할하는 기준으로 사용되는 척도입니다. 이는 불순도(Impurity)의 감소량을 측정하여, 특정한 속성을 기준으로 데이터를 분할할 때 얼마나 효과적인지를 평가합니다.

개념

정보 이득은 다음과 같은 과정을 통해 계산됩니다:

1. 전체 데이터의 불순도 계산:

- 데이터를 하나의 그룹으로 간주하고, 이 그룹의 불순도를 계산합니다. 일반적으로 엔트로피(Entropy)나 지니 지수(Gini Index)를 사용합니다.

2. 분할 후 불순도 계산:

- 특정 속성을 기준으로 데이터를 두 개 이상의 그룹으로 분할합니다.
- 각 그룹의 불순도를 계산하고, 그룹의 크기에 비례하여 가중 평균을 구합니다.

3. 정보 이득 계산:

- 전체 데이터의 불순도에서 분할 후의 가중 평균 불순도를 뺀 값을 정보 이득이라고 합니다.

공식

- 전체 데이터의 불순도: $I(S)$

- 분할 후의 가중 평균 불순도: $I(S|A)$

정보 이득 IG 는 다음과 같이 계산됩니다:

$$IG(S, A) = I(S) - I(S|A)$$

여기서:

- S 는 전체 데이터셋
- A 는 분할에 사용되는 속성

In [51]: `Image('175.png')`

Out[51]: 불순도 척도

- **엔트로피(Entropy):** 불확실성을 측정하는 척도입니다. 엔트로피가 높을수록 데이터가 혼합되어 있음을 의미합니다.

$$H(S) = - \sum_{i=1}^n p_i \log_2(p_i)$$

여기서 p_i 는 클래스 i 의 비율입니다.

- **지니 지수(Gini Index):** 두 개의 임의의 샘플이 서로 다른 클래스에 속할 확률을 나타냅니다.

$$G(S) = 1 - \sum_{i=1}^n (p_i)^2$$

여기서 p_i 는 클래스 i 의 비율입니다.

In [52]: `Image('176.png')`

Out[52]: **예시**

예를 들어, 주어진 데이터셋이 다음과 같다고 가정합시다:

Feature	Class
A	0
A	0
B	1
B	1

전체 데이터셋의 엔트로피는 다음과 같이 계산할 수 있습니다:

$$H(S) = - \left(\frac{2}{4} \log_2 \frac{2}{4} + \frac{2}{4} \log_2 \frac{2}{4} \right) = 1$$

만약 속성 "Feature"를 기준으로 데이터를 분할한다면:

- "Feature = A": [0, 0]
- "Feature = B": [1, 1]

각 그룹의 엔트로피는 0입니다 (완전히 순수하기 때문입니다).

분할 후의 가중 평균 엔트로피는:

$$H(S|\text{Feature}) = \frac{2}{4} \times 0 + \frac{2}{4} \times 0 = 0$$

따라서 정보 이득은:

$$IG(S, \text{Feature}) = 1 - 0 = 1$$

이 예시에서 정보 이득이 1이라는 것은, 속성 "Feature"를 기준으로 데이터를 분할하는 것이 매우 유용함을 나타냅니다. 이는 분할 전후의 불순도 차이가 크다는 것을 의미합니다.

```
In [53]: '''
함수는 데이터프레임 `df`의 특정 열 `col`을 기준으로
분할하여 정보 이득(Information Gain, IG)을 계산합니다.
정보 이득은 불순도(엔트로피 또는 지니 지수 등)의
감소를 측정하는 방법입니다. 이를 통해 열 `col`을
기준으로 데이터를 분할할 때 얼마나 유용한지를 평가합니다.

### 전체 함수 설명
함수 `ig`는 다음과 같은 입력을 받습니다:
- `df`: 데이터프레임
- `col`: 분할할 열 이름
- `imp_func`: 불순도를 계산하는 함수 (예: 엔트로피 함수)

출력으로는 다음 정보를 포함하는 시리즈를 반환합니다:
- `ig`: 정보 이득
- `imp1`: 좌측 영역의 불순도
- `imp2`: 우측 영역의 불순도
- `n1`: 좌측 영역의 데이터 수
- `n2`: 우측 영역의 데이터 수

### 각 부분 설명
1. 고유값 및 분류 경계 계산:
  - `u`는 `col` 열의 고유값을 정렬한 시리즈입니다.
  - `b`는 분류 경계로, 고유값의 중간 지점을 사용합니다.
    `[np.inf]`와 `[-np.inf]`는 경계 밖의 값들을
    포함하기 위함입니다.
  - `n`은 데이터프레임 `df`의 총 데이터 수입니다.
2. 빈도의 누계 계산:
  - `pd.crosstab`은 `col`을 기준으로
    `Survived` 열의 빈도를 계산합니다.
```

```

- `cumsum()`은 누적 합계를 계산합니다.
- `vcnt`는 `df_cnt`의 마지막 행의 값을 배열로
  저장합니다. 이는 전체 데이터의 `Survived`
  열의 빈도를 나타냅니다.
3. 전체 불순도 계산:
- `imp_t`는 전체 데이터의 불순도를 계산합니다.
- `df_cnt = df_cnt.iloc[:-1]`는 마지막 행을 제외합니다.
  이는 누적 합계의 중복 계산을 방지합니다.
4. 서브 함수 `ig_sub` 정의:
- `ig_sub`는 각 구간의 불순도와 가중치가 적용된
  불순도를 계산합니다.
- `x`는 특정 구간의 빈도 값입니다.
- `s`는 해당 구간의 총 데이터 수입니다.
- `imp`는 구간의 불순도입니다.
- `imp_by_weight`는 가중치가 적용된 불순도입니다.
5. 좌측 영역 및 우측 영역의 불순도 계산:
- `df_left`는 각 구간의 좌측 영역의 불순도를 계산합니다.
- `df_right`는 각 구간의 우측 영역의 불순도를 계산합니다.
6. 결과 반환:
- `imp_t - df_left['imp_by_weight']
  - df_right['imp_by_weight']`는 정보 이득을 계산합니다.
- `df_left['imp']`, `df_right['imp']`는
  좌측 및 우측 영역의 불순도입니다.
- `df_left['n']`, `df_right['n']`는
  좌측 및 우측 영역의 데이터 수입니다.
- `pd.concat`는 이 모든 결과를 하나의
  데이터프레임으로 합칩니다.

```

전체 동작 과정 요약

- 데이터프레임 `df`의 `col` 열을 기준으로
고유값을 정렬하고, 분류 경계를 계산합니다.
- 각 분류 경계에 대해 `Survived` 열의
빈도 누적 합계를 계산합니다.
- 전체 데이터의 불순도를 계산합니다.
- 각 분류 경계별로 좌측 및 우측 영역의
불순도와 가중치가 적용된 불순도를 계산합니다.
- 정보 이득을 계산하고, 좌측 및 우측 영역의
불순도와 데이터 수를 반환합니다.

이 함수는 주어진 열을 기준으로 데이터를 분할할 때,

```

분할의 유용성을 평가하는 데 사용될 수 있습니다.
'''

def ig(df, col, imp_func):
    """
    불순도 차감을 계산합니다.
    위 과정을 종합하여 하나의 함수로 정리했습니다.
    Parameters:
        df: pd.DataFrame
            데이터프레임
        col: str
            대상 컬럼
        imp_func: function
            불순도 계산 함수
    Returns:
        pd.Series
        ig: 불순도 차감, imp1: 좌측 영역의 불순도,
        imp2: 우측 영역의 불순도,
        n1: 좌측 영역의 데이터수,
        n2: 우측 영역의 데이터수
    """

    # col 컬럼의 유일값을 구하고, 정렬합니다.
    u = pd.Series(df[col].unique()).sort_values()
    # 분류 경계를 구합니다.
    b = np.array([-np.inf] + ((u + u.shift(1)) / 2)\
                  .dropna().tolist() + [np.inf])

    n = len(df)
    # 분류 경계점별 불순도를 구하기 위해 구간별
    # 대상 변수(Survived)의 빈도의 누계를 구합니다.
    df_cnt = pd.crosstab(
        index = pd.cut(df[col], bins=b, right=True),
        columns = df['Survived']
    ).cumsum()
    vcnt = df_cnt.iloc[-1].values
    # 분류 경계별 불순도를 구합니다.
    imp_t = imp_func(vcnt / n)
    df_cnt = df_cnt.iloc[:-1]

    def ig_sub(x):
        """

```

```

불순도를 구합니다.
Parameters:
    x: array-like
        데이터
Returns:
    pd.Series
    imp: 불순도, imp_by_weight: 가중치 * 불순도, n: 데이터 수
"""
s = x.sum()
imp = imp_func(x / s)
return pd.Series([imp, imp * s / n, s],
                  index=['imp', 'imp_by_weight', 'n'])

# 우측 영역의 불순도를 구합니다.
df_left = df_cnt.apply(ig_sub, axis=1)
# 좌측영역의 불순도를 구합니다.
df_right = (vcnt - df_cnt).apply(ig_sub, axis=1)
return pd.concat([
    (imp_t - df_left['imp_by_weight'] \
     - df_right['imp_by_weight']).rename('ig'),
    df_left['imp'].rename('imp1'),
    df_right['imp'].rename('imp2'),
    df_left['n'].rename('n1'),
    df_right['n'].rename('n2')
], axis=1)

```

In [54]: '''
 코드는 `df\_titanic` 데이터프레임에서 `Pclass\_1` 열을
 기준으로 정보 이득(Information Gain, IG)을 계산하고,
 그 중 최대 정보 이득을 보인 분기점을 찾는 과정입니다.

1. 정보 이득 계산:

- `ig(df\_titanic, 'Pclass\_1', entropy)`는
`df\_titanic` 데이터프레임에서 `Pclass\_1` 열을
기준으로 정보 이득을 계산합니다.
- `entropy` 함수는 불순도를 계산하기 위해 사용됩니다.
- 반환된 `df\_imp`는 각 분기점에서의 정보 이득,
좌측 및 우측 영역의 불순도, 데이터 수 등을
포함하는 데이터프레임입니다.

2. 최대 정보 이득을 보인 분기점 찾기:

- `df_imp['ig'].idxmax()`는 `df_imp` 데이터프레임의 `ig` 열에서 최대 값을 가지는 인덱스를 찾습니다.
- `df_imp.loc[최대 인덱스]`는 해당 인덱스에 해당하는 행을 반환합니다. 이 행은 최대 정보 이득을 보인 분기점의 정보를 포함합니다.

```
### 예시 데이터프레임
가상의 `df_imp` 데이터프레임이
다음과 같다고 가정해 보겠습니다:
'''
Image('177.png')
```

Out[54]:

index	ig	imp1	imp2	n1	n2
Parch <= 0	0.1	0.5	0.4	100	200
Parch <= 1	0.2	0.4	0.3	150	150
Parch <= 2	0.15	0.45	0.35	120	180

In [55]:

```
'''
여기서 최대 정보 이득을 보이는 행은 `Parch <= 1`이 됩니다.
따라서, `df_imp['ig'].idxmax()`는 `Parch <= 1`의
인덱스를 반환하고, `df_imp.loc[df_imp['ig'].idxmax()]`는
다음과 같은 시리즈를 반환합니다:
```

```
ig      0.2
imp1    0.4
imp2    0.3
n1      150
n2      150
Name: Parch <= 1, dtype: float64
```

```
### 의미
이 결과는 `Pclass_1` 열을 기준으로 데이터를 분할할 때,
분할의 유용성을 평가한 후, 최대 정보 이득을
보이는 분기점의 정보를 제공합니다.
최대 정보 이득을 보이는 분기점을 사용하면,
```

의사결정트리와 같은 모델이 데이터를 가장 잘 분할할 수 있습니다. 이 분기점에서의 불순도 감소가 가장 크기 때문에, 해당 분기점을 선택하는 것이 분류 성능을 높이는 데 도움이 됩니다.

```
'''
df_imp = ig(df_titanic, 'Pclass_1', entropy)
# 최대 불순도 감소를 보인 분기점을 구합니다.
df_imp.loc[df_imp['ig'].idxmax()]
'''
```

```
Out[55]: ig      0.057796
         imp1    0.887537
         imp2    0.950956
         n1      675.000000
         n2      216.000000
         Name: (-inf, 0.5], dtype: float64
```

```
In [56]: '''
코드는 여러 변수에서 최대 정보 이득을 제공하는
최적의 분기점을 찾아내고, 이를 데이터프레임으로
정리하는 과정을 수행합니다.

1. 초기화:
- `max_ig` 리스트를 초기화합니다.
이 리스트는 각 변수에서 최대 정보 이득을 제공하는
최적의 분기점을 저장하는 데 사용됩니다.
2. 각 변수에 대한 최적의 분기점 찾기:
- `X_all` 리스트의 각 변수 `i`에 대해 반복합니다.
- `ig(df_titanic, i, gini)` 함수 호출을 통해
변수 `i`에 대한 정보 이득을 계산합니다.
여기서 `gini` 함수는 불순도 계산에 사용됩니다.
- `df_imp['ig'].idxmax()`를 사용하여 정보 이득이
최대인 인덱스를 찾습니다.
- `df_imp.loc[midx]`는 최대 정보 이득을 보이는
분기점의 정보를 포함하는 행을 가져옵니다.
- `.rename(i + '<=' + str(midx.right))`는 해당 행의
이름을 `i<=분기점` 형식으로 변경합니다.
`midx.right`는 분기점의 우측 경계를 나타냅니다.
- 이 행을 `max_ig` 리스트에 추가합니다.
3. 결과를 데이터프레임으로 정리:
- `max_ig` 리스트의 모든 요소를 열로 하여
'''
```

하나의 데이터프레임으로 병합합니다.

- ``T``는 행과 열을 전치(**transpose**)하여 각 변수별로 최적의 분기점 정보를 행으로 정리합니다.

...

`Image('178.png')`

Out[56]:

예시 데이터

가상의 예시를 통해 설명해 보겠습니다.

입력 데이터프레임 `df\_titanic`

Pclass_1	Pclass_2	Pclass_3	...	Survived
1	0	0	...	1
0	1	0	...	0
0	0	1	...	1
...	...	...	...	...

중간 결과 `df\_imp`

index	ig	imp1	imp2	n1	n2
Parch <= 0	0.1	0.5	0.4	100	200
Parch <= 1	0.2	0.4	0.3	150	150
Parch <= 2	0.15	0.45	0.35	120	180

최종 결과 `df\_ig`

	ig	imp1	imp2	n1	n2
Pclass_1 <= 0.5	0.2	0.4	0.3	150	150
Pclass_2 <= 1.5	0.15	0.45	0.35	120	180

Pclass_3 <= 2.5	0.1	0.5	0.4	100	200
-----------------	-----	-----	-----	-----	-----

```
In [25]: '''
### 최종 데이터프레임의 의미
- `df_ig` 데이터프레임의 각 행은 `X_all` 리스트의
  각 변수에 대한 최적의 분기점을 나타냅니다.
- `ig` 열은 정보 이득(불순도 감소)을 나타내고,
  `imp1`과 `imp2`는 각각 좌측과 우측 영역의
  불순도를 나타냅니다.
- `n1`과 `n2`는 좌측과 우측 영역의 데이터 수를 나타냅니다.
- 각 행의 이름은 변수명과 최적의 분기점을 결합한 형태로,
  예를 들어 `Pclass_1 <= 0.5`는
  `Pclass_1` 변수가 0.5 이하일 때의 정보를 나타냅니다.

이 과정은 각 변수에 대해 데이터를 분할할 최적의 분기점을
찾아내어, 이를 이용해 의사결정트리 모델링에서 효과적인
분할 기준을 설정하는 데 사용될 수 있습니다.
'''

max_ig = list()
# 모든 변수에서 최적의 분기점(불순도 감소가 가장 큰)을 구합니다.
for i in X_all:
    df_imp = ig(df_titanic, i, gini)
    midx = df_imp['ig'].idxmax()
    max_ig.append(
        df_imp.loc[midx].rename(i + '<=' + str(midx.right))
    )

df_ig = pd.concat(max_ig, axis=1).T
df_ig
```

Out[25]:

	ig	imp1	imp2	n1	n2
Age<=6.5	0.011283	0.418289	0.464149	47.0	844.0
SibSp<=0.5	0.006350	0.452194	0.497746	608.0	283.0
Parch<=0.5	0.010278	0.451114	0.499724	678.0	213.0
Fare<=10.481	0.042584	0.317157	0.499993	339.0	552.0
Sex<=0.5	0.139648	0.306444	0.382835	577.0	314.0
Pclass_1<=0.5	0.038665	0.424094	0.466392	675.0	216.0
Pclass_2<=0.5	0.004122	0.461179	0.498523	707.0	184.0
Pclass_3<=0.5	0.049138	0.493387	0.367246	400.0	491.0
Embarked_C<=0.5	0.013389	0.451576	0.494260	723.0	168.0
Embarked_Q<=0.5	0.000006	0.472759	0.475628	814.0	77.0
Embarked_S<=0.5	0.010598	0.499992	0.448164	245.0	646.0

In [57]:

```
'''
코드는 `df_ig` 데이터프레임에서 정보 이득
(Information Gain,IG)이 최대인 행을 선택하는 과정입니다.

1. 최대 정보 이득 찾기:
- `df_ig` 데이터프레임의 `ig` 열에서 최대 값을 찾습니다.
이 값은 데이터프레임에서 정보 이득이
가장 큰 값을 나타냅니다.
2. 최대 정보 이득과 일치하는 행 필터링:
- `df_ig['ig']`의 각 값이 최대 정보 이득 값과
같은지 비교합니다.
- 이 비교 결과는 불리언 시리즈로 반환됩니다.
최대 정보 이득과 일치하는 위치는 `True`,
나머지는 `False` 값을 가집니다.
3. 최대 정보 이득을 가진 행 선택:
- `loc` 인덱서를 사용하여 `df_ig` 데이터프레임에서
정보 이득이 최대인 행을 선택합니다.
- `df_ig['ig'] == df_ig['ig'].max()` 조건에
```

... 맞는 행만 선택하여 반환합니다.

...

Image('179.png')

Out[57]:

예시 데이터

가상의 예시를 통해 설명해 보겠습니다.

입력 데이터프레임 `df_ig`

	ig	imp1	imp2	n1	n2
Pclass_1 <= 0.5	0.2	0.4	0.3	150	150
Pclass_2 <= 1.5	0.15	0.45	0.35	120	180
Pclass_3 <= 2.5	0.1	0.5	0.4	100	200

최대 정보 이득 찾기

- `df_ig['ig'].max()`의 결과는 `0.2`입니다.

필터링 결과

- `df_ig['ig'] == df_ig['ig'].max()`의 결과는 다음과 같습니다:

	ig
Pclass_1 <= 0.5	True
Pclass_2 <= 1.5	False
Pclass_3 <= 2.5	False

최종 선택된 행

- `df_ig.loc[df_ig['ig'] == df_ig['ig'].max()]`의 결과는 다음과 같습니다:

	ig	imp1	imp2	n1	n2
Pclass_1<=0.5	0.2	0.4	0.3	150	150

```
In [26]: '''
### 최종 데이터프레임의 의미
이 결과는 `df_ig` 데이터프레임에서 정보 이득이
최대인 행을 나타냅니다. 즉, 주어진 변수들 중에서
특정 변수와 그 분기점이 데이터 분할에
가장 유용한 기준임을 보여줍니다.
이 정보는 의사결정트리와 같은 모델링에서
데이터 분할 기준을 설정하는 데 사용될 수 있습니다.
'''
```

```
df_ig.loc[df_ig['ig'] == df_ig['ig'].max()]
```

```
Out[26]:
```

	ig	imp1	imp2	n1	n2
Sex<=0.5	0.139648	0.306444	0.382835	577.0	314.0

```
In [58]: '''
코드는 Scikit-learn 라이브러리를 사용하여
결정 트리(Decision Tree) 모델을 생성하고
시각화하는 과정을 보여줍니다.

1. 그래프 크기 설정:
- 그래프의 크기를 설정합니다. `figsize=(8, 2)`는
  가로 8인치, 세로 2인치 크기를 의미합니다.
2. 결정 트리 모델 생성:
- `DecisionTreeClassifier` 객체를 생성합니다.
- `max_depth=1`은 트리의 최대 깊이를 1로 설정합니다.
  이는 트리가 하나의 분기점만 가지도록 제한합니다.
3. 모델 학습:
- `fit` 메서드를 사용하여 결정 트리 모델을 학습시킵니다.
```

```
- `df_titanic[X_all]`는 독립 변수(특성)들을 나타내고,
  `df_titanic['Survived']`는
  종속 변수(타겟)를 나타냅니다.
4. 트리 시각화:
- `plot_tree` 함수를 사용하여 학습된
  결정 트리를 시각화합니다.
- `feature_names=X_all`은 각 노드에 표시될
  특성 이름을 지정합니다.
- `plt.show()`는 그래프를 화면에 표시합니다.
...
```

Image('180.png')

Out[58]: **트리 구조의 의미**

예시 데이터프레임 `df\_titanic`

Pclass_1	Pclass_2	Pclass_3	Age	SibSp	Parch	Fare	Embarked_C	Embarked_Q	Embarked_S
0	1	0	22	1	0	7.25	0	0	1
1	0	0	38	1	0	71.28	1	0	0
0	1	0	26	0	0	7.92	0	0	1
...	...	...	...	...	...	...	...	...	...

In [59]: Image('181.png')

Out[59]:

Embarked_C	Embarked_Q	Embarked_S	Sex
0	0	1	1
1	0	0	1
0	0	1	0
...	...	...	...

In [27]:

```
'''
### 시각화 결과
- 트리 시각화는 하나의 분기점과
  두 개의 리프 노드로 구성됩니다.
- 각 노드에는 다음 정보가 포함됩니다:
  - 분기 기준(예: `Pclass_1 <= 0.5`)
  - 각 클래스의 표본 수
  - 지니 불순도(Gini impurity)
  - 대부분 클래스의 예측값

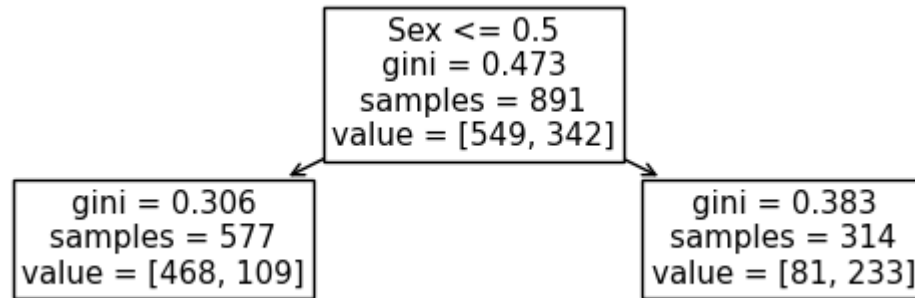
### 코드의 전체적인 의미
이 코드는 단순한 결정 트리 모델을 생성하여
주어진 데이터에서 생존 여부를 예측하고,
그 결과를 시각화하는 과정을 보여줍니다.
트리의 최대 깊이를 1로 설정하여 단일 분기점만을
가지므로, 트리 구조는 매우 단순하게 나타납니다.
이를 통해 어떤 특성이 가장 중요한지,
그 특성의 어떤 값이 분류에 중요한 역할을
하는지 쉽게 이해할 수 있습니다.
'''

from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree

plt.figure(figsize=(8, 2))

clf_dt = DecisionTreeClassifier(max_depth=1)
clf_dt.fit(df_titanic[X_all], df_titanic['Survived'])
```

```
plot_tree(clf_dt, feature_names=X_all)  
plt.show()
```



```
In [18]: 0.473 - 0.306 * 577 / 891 - 0.383 * 314 / 891
```

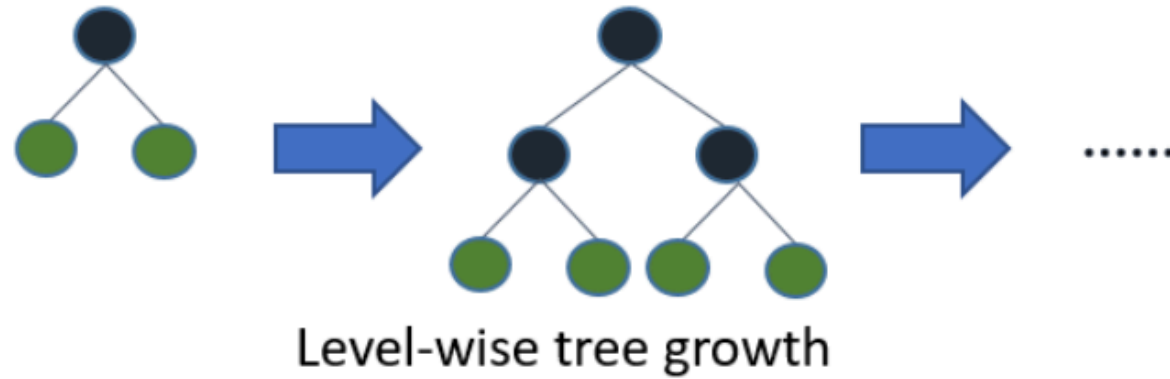
```
Out[18]: 0.13986419753086418
```

```
In [60]: Image('182.png')
```


Out[60]: **나무의 성장 방법**

- 레벨기준(Level(Depth)-Wise)

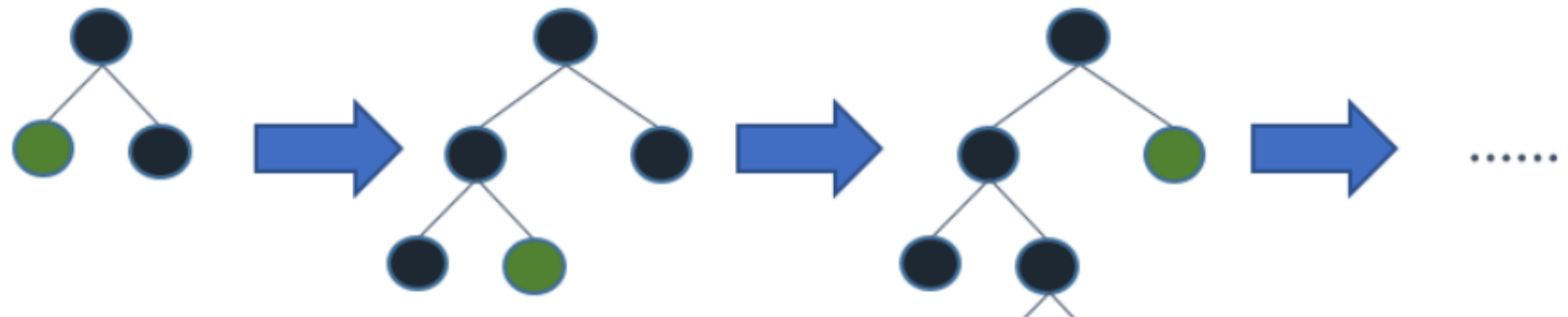
나무의 레벨을 기준으로 나무를 성장 시킵니다.



※ 이미지 출처: <https://lightgbm.readthedocs.io/en/latest/Features.html>

- 성능기준(Leaf-Wise, Best-First)

나무의 불순도 차감을 기준으로 나무를 성장 시킵니다.





Leaf-wise tree growth

※ 이미지 출처: <https://lightgbm.readthedocs.io/en/latest/Features.html>

sklearn.tree.DecisionTree(Classifier, Regressor) 에서는

max_leaf_nodes 설정을 하면 Best-First 방식으로 동작합니다.

sklearn.tree.DecisionTree(Classifier, Regressor) 매뉴얼 발췌

```
max_leaf_nodes: int, default=None
    Grow a tree with max_leaf_nodes in best-first fashion.
    Best nodes are defined as relative reduction in impurity.
    If None then unlimited number of leaf nodes.
```

[Ex.3]

df_titanic에서 입력 변수를 X_all 대상 변수를 Survived로 했을 때,
레벨 기준 성장 방법과 성능 기준 성장 방법의 차이점을 확인해봅니다.

In [62]:

```
'''
코드는 결정 트리 모델을 생성하고, 학습시키고,
시각화하는 과정을 보여줍니다. 트리의 최대 깊이를
3으로 설정하여 더 복잡한 트리를 생성합니다.

1. 그래프 크기 설정:
- 그래프의 크기를 설정합니다. `figsize=(16, 5)`는
  가로 16인치, 세로 5인치 크기를 의미합니다.
- `fig = plt.figure()`는 새로운 그림을 생성하고
  이를 `fig` 객체에 할당합니다.
  이는 그래프의 크기 및 기타 속성을
```

설정하는 데 사용됩니다.

2. 결정 트리 모델 생성:

- ``DecisionTreeClassifier`` 객체를 생성합니다.
- ``max_depth=3``는 트리의 최대 깊이를 3으로 설정합니다.
이는 트리가 최대 3개의 분기점을
가질 수 있음을 의미합니다.

3. 모델 학습:

- ``fit`` 메서드를 사용하여 결정 트리 모델을
학습시킵니다.
- ``df_titanic[X_all]``는 독립 변수(특성)들을
나타내고, ``df_titanic['Survived']``는
종속 변수(타겟)를 나타냅니다.
- 모델은 주어진 데이터를 기반으로
최적의 분기 기준을 찾아 트리 구조를 형성합니다.

4. 트리 시각화:

- ``plot_tree`` 함수를 사용하여 학습된
결정 트리를 시각화합니다.
- ``feature_names=X_all``은 각 노드에
표시될 특성 이름을 지정합니다.

트리 구조의 의미

트리 시각화는 여러 분기점과 리프 노드로 구성됩니다.

각 노드에는 다음 정보가 포함됩니다:

- 분기 기준:
예를 들어, ``Pclass_1 <= 0.5``는 해당 특성에
따라 데이터를 분류하는 기준을 나타냅니다.
- 지니 불순도(Gini impurity):
노드 내 데이터의 혼합도를 나타내는 지표입니다.
값이 낮을수록 노드 내의 데이터가
더 잘 분류된 상태를 의미합니다.
- 각 클래스의 표본 수: 노드에 포함된
각 클래스의 데이터 수를 나타냅니다.
- 대부분 클래스의 예측값: 해당 노드에서
예측되는 클래스 값입니다.
- ...

`Image('183.png')`

Out[62]: 예시 데이터프레임 `df\_titanic`

Pclass_1	Pclass_2	Pclass_3	Age	SibSp	Parch	Fare	Embarked_C	Embarked_Q	Embarked_S
0	1	0	22	1	0	7.25	0	0	1
1	0	0	38	1	0	71.28	1	0	0
0	1	0	26	0	0	7.92	0	0	1
...	...	...	...	...	...	...	...	...	...

```
In [19]: '''
### 트리 시각화 결과
트리 시각화는 여러 분기점과 리프 노드를 포함하는
더 복잡한 구조를 보여줍니다. 깊이가 3이므로 트리는
최대 3개의 분기점까지 가질 수 있으며,
이는 보다 정교한 데이터 분류를 가능하게 합니다.

- 최상위 노드 (루트 노드):
  전체 데이터를 첫 번째 분기 기준으로 나눕니다.
- 중간 노드:
  루트 노드의 각 분기에서 데이터의
  하위 집합을 추가 분기합니다.
- 리프 노드:
  더 이상 분기하지 않고 최종 예측을
  나타내는 노드입니다.

### 코드의 전체적인 의미
이 코드는 주어진 데이터에서 결정 트리 모델을 학습시키고,
이를 시각화하여 트리 구조를 통해 어떻게 데이터를 분류하는지
이해할 수 있도록 합니다. 트리의 깊이를 3으로 설정하여,
더 복잡한 분류 기준을 통해 데이터의 패턴을 보다
세밀하게 분석할 수 있습니다.
시각화된 트리는 데이터의 분포와
각 특성의 중요성을 시각적으로 보여줍니다.
'''
```

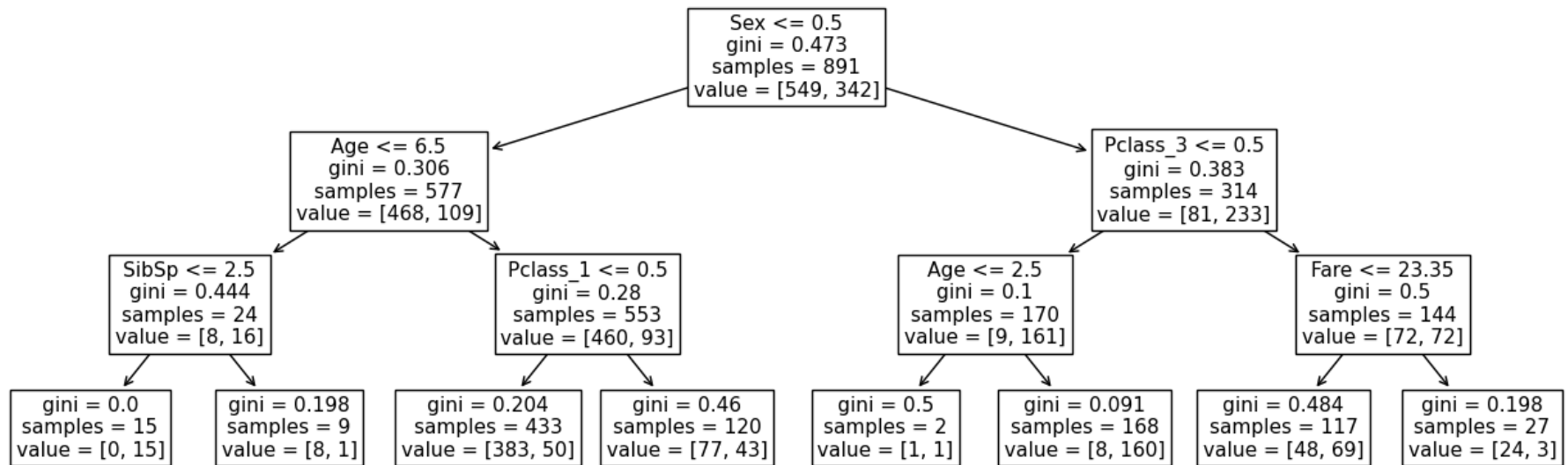
```

from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree

fig = plt.figure(figsize=(16, 5))

clf_dt = DecisionTreeClassifier(max_depth=3, random_state=123)
clf_dt.fit(df_titanic[X_all], df_titanic['Survived'])
plot_tree(clf_dt, feature_names=X_all)
plt.show()

```



In [63]: `'''`

코드는 결정 트리 모델을 생성하고, 학습시키고, 시각화하는 과정을 보여줍니다. 트리의 최대 잎 노드 수를 8로 제한하여 트리의 복잡성을 조절합니다.

1. 결정 트리 모델 생성:

- `DecisionTreeClassifier` 객체를 생성합니다.
- `max_leaf_nodes=8`는 트리의 최대 잎 노드 수를 8로 설정합니다. 이는 트리가 최대 8개의 리프 노드를 가질 수 있음을 의미합니다.
- 트리의 깊이 대신 잎 노드 수를 제한함으로써

트리의 복잡성을 조절합니다.

2. 모델 학습:

- ``fit`` 메서드를 사용하여 결정 트리 모델을 학습시킵니다.
- ``df_titanic[X_all]``는 독립 변수(특성)들을 나타내고, ``df_titanic['Survived']``는 종속 변수(타겟)를 나타냅니다.
- 모델은 주어진 데이터를 기반으로 최적의 분기 기준을 찾아 트리 구조를 형성합니다.

3. 트리 시각화:

- ``plot_tree`` 함수를 사용하여 학습된 결정 트리를 시각화합니다.
- ``feature_names=X_all``은 각 노드에 표시될 특성 이름을 지정합니다.

4. 레이아웃 조정:

- ``plt.tight_layout()``은 그래프의 레이아웃을 자동으로 조정하여, 요소들이 서로 겹치지 않도록 합니다. 이는 그래프의 가독성을 높이는 데 도움이 됩니다.

트리 구조의 의미

트리 시각화는 여러 분기점과 리프 노드로 구성됩니다.

각 노드에는 다음 정보가 포함됩니다:

- 분기 기준:
예를 들어, ``Pclass_1 <= 0.5``는 해당 특성에 따라 데이터를 분류하는 기준을 나타냅니다.
- 지니 불순도 (Gini impurity):
노드 내 데이터의 혼합도를 나타내는 지표입니다.
값이 낮을수록 노드 내의 데이터가 더 잘 분류된 상태를 의미합니다.
- 각 클래스의 표본 수:
노드에 포함된 각 클래스의 데이터 수를 나타냅니다.
- 대부분 클래스의 예측값:
해당 노드에서 예측되는 클래스 값입니다.

...

`Image('184.png')`

Out[63]: 예시 데이터프레임 `df\_titanic`

Pclass_1	Pclass_2	Pclass_3	Age	SibSp	Parch	Fare	Embarked_C	Embarked_Q	Embarked_S
0	1	0	22	1	0	7.25	0	0	1
1	0	0	38	1	0	71.28	1	0	0
0	1	0	26	0	0	7.92	0	0	1
...	...	...	...	...	...	...	...	...	...

```
In [20]: '''
### 트리 시각화 결과

트리 시각화는 최대 8개의 리프 노드를 포함하는 구조를
보여줍니다. 이는 보다 제한된 복잡성을 가지며,
과적합을 방지하고 모델의 해석 가능성을 높입니다.

- 최상위 노드 (루트 노드):
  전체 데이터를 첫 번째 분기 기준으로 나눕니다.
- 중간 노드:
  루트 노드의 각 분기에서 데이터의
  하위 집합을 추가 분기합니다.
- 리프 노드:
  더 이상 분기하지 않고 최종 예측을
  나타내는 노드입니다.

### 코드의 전체적인 의미

코드는 주어진 데이터에서 결정 트리 모델을 학습시키고,
이를 시각화하여 트리 구조를 통해 어떻게 데이터를
분류하는지 이해할 수 있도록 합니다.
트리의 최대 잎 노드 수를 8로 제한하여,
모델의 복잡성을 조절하고, 과적합을 방지합니다.
시각화된 트리는 데이터의 분포와 각 특성의 중요성을
시각적으로 보여줍니다. `plt.tight_layout()`을 사용하여
```

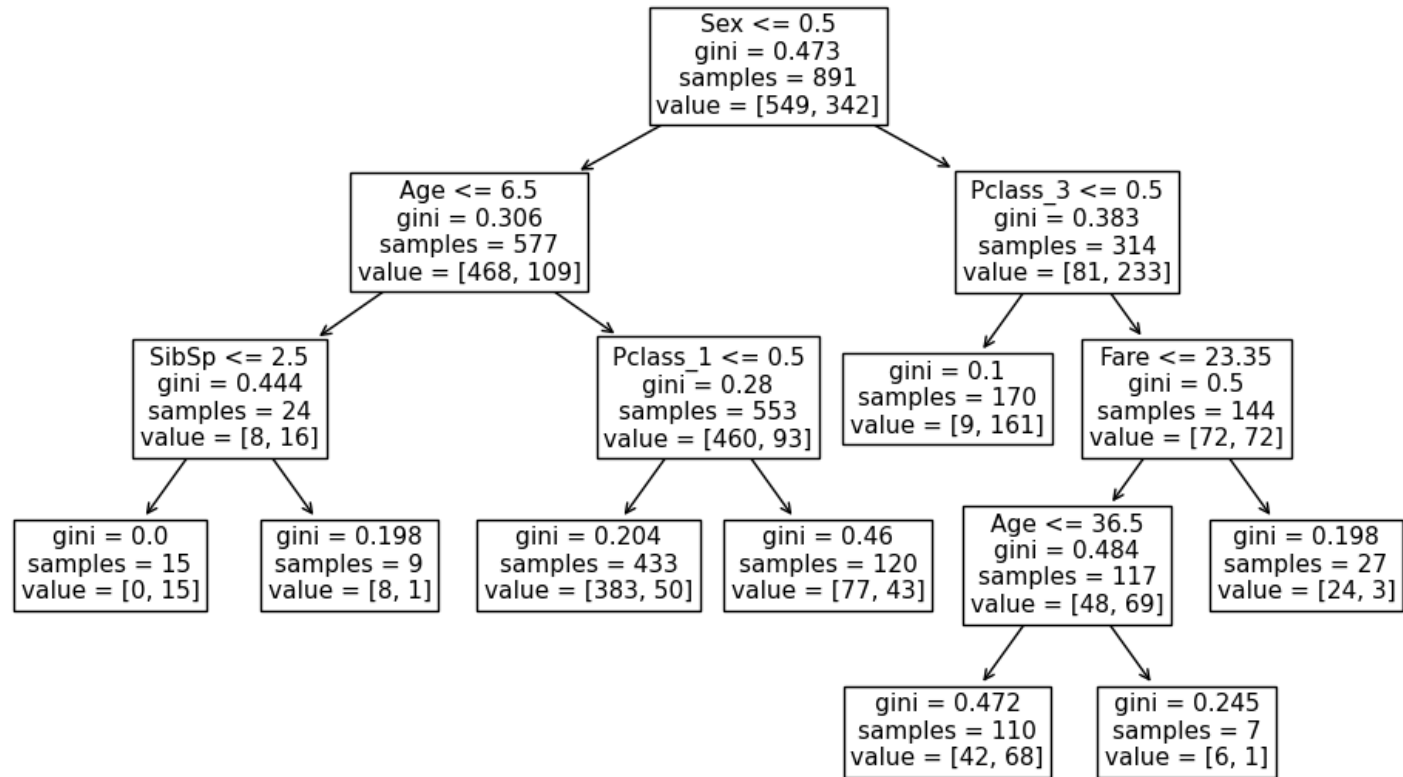
그래프 요소들이 겹치지 않도록 레이아웃을 조정하여
가독성을 높입니다.

'''

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree

fig = plt.figure(figsize=(12, 7))
clf_dt = DecisionTreeClassifier(max_leaf_nodes=8)
clf_dt.fit(df_titanic[X_all], df_titanic['Survived'])
plot_tree(clf_dt, feature_names=X_all)

plt.tight_layout()
plt.show()
```

In [64]: Image('185.png')

Out[64]: 장단점 비교

방법	장점	단점
Level-wise	구조가 단순하여 해석이 쉽습니다. 계산 비용이 Leaf-wise에 비해 낮습니다. 각 레벨에서 독립적으로 분기를 하므로 병렬화에 용이합니다.	Leaf-wise에 비해 비효율적인 분할을 지닐 가능성이 높습니다.
Leaf-wise	효과가 높은 것을 우선 분기하여 최적의 구조를 찾을 가능성이 높습니다.	계산 비용이 Level-wise에 비해 높습니다. 과적합 가능성이 Level-wise에 비해 높습니다. 최선의 노드를 선택 후에 분기를 위한 연산이 일어나는 의존적 구조라 병렬화가 어렵습니다.

In [65]: Image('186.png')

Out[65]:

4. 가지치기(Pruning)

의사 결정 나무는 데이터에 맞추어 가설 공간을 키워 갈수 있는 구조입니다.

크기의 제한이 없다면 높은 모델의 복잡도로 인해 지나치게 높은 분산으로 인한 과적합이 발생합니다.

의사 결정 나무 학습이전에 크기의 제한을 두는 사전가지치기(Pre-pruning)과

학습후에 크기를 줄이는 사후 가지치기(Post-pruning)가 있습니다.

sklearn.tree.DecisionTreeClassifier와 DecisionTreeRegressor

주요 하이퍼 파라미터

하이퍼 파라미터	기능
max_depth	나무가 성장할 수 있는 최대 깊이
max_leaf_nodes	최대 리프 노드 수, Leaf-wise 방식으로 성장하게 됩니다.
min_samples_split	분기를 위한 최소 샘플 수
min_samples_leaf	리프 노드의 최소 샘플 수
min_impurity_decrease	분기에 필요한 최소 불순도 차감
min_impurity_split	분기에 필요한 최소 불순도

예제에 활용하기 위해 df_titanic에서 80%는

학습 데이터 df_train으로 20%는 평가데이터

df_test로 Survived 비율이 동일하게 나눕니다.

```
In [21]: from sklearn.model_selection import train_test_split

df_train, df_test
```

```
= train_test_split(df_titanic, train_size=0.8,
                    stratify=df_titanic['Survived'],
                    random_state=123)
```

[Ex.4]

max_depth 에 따라 학습과 평가의 성능이 어떻게 변하는지 확인합니다.

입력 변수는 X_all 리스트에 있는 변수이고, 대상 변수는 Survived 입니다.

성능 측정 지표는 정확도(Accuracy) 입니다.

```
max_depths = np.arange(2, 13)
```

In [66]:

```
'''
코드는 다양한 `max_depth` 값에 대해 결정 트리 모델을
학습시키고, 각 `max_depth` 값에서의 학습 데이터와
테스트 데이터에 대한 정확도를 계산하여 리스트에
저장합니다. 이를 통해 모델의 복잡도에 따른
성능 변화를 분석할 수 있습니다.

1. `max_depths` 설정:
- `max_depths`는 2부터 12까지의 정수로 이루어진
  배열입니다. 이는 트리의 최대 깊이를 설정할
  값들의 범위를 나타냅니다.
- `np.arange(2, 13)`는 2부터 12까지의
  정수를 생성합니다.

2. 정확도 저장을 위한 리스트 초기화:
- `acc_train`과 `acc_test`는 각각 학습 데이터와
  테스트 데이터의 정확도를 저장할 리스트입니다.

3. 루프를 통한 모델 학습 및 평가:
- `for max_depth in max_depths`:
  `max_depths` 배열의 각 값에 대해
  반복문을 실행합니다.
- `clf_dt = DecisionTreeClassifier
  (max_depth=max_depth, random_state=123)`:
  현재 `max_depth` 값으로 결정 트리
  모델을 생성합니다.
'''
```

```
- `clf_dt.fit(df_train[X_all], df_train['Survived'])`:
    학습 데이터를 사용하여 모델을 학습시킵니다.
- `acc_test.append(accuracy_score(df_test['Survived'],
    clf_dt.predict(df_test[X_all])))`:
    테스트 데이터에 대한 예측을 수행하고,
    정확도를 계산하여 `acc_test` 리스트에 추가합니다.
- `acc_train.append(accuracy_score(df_train['Survived'],
    clf_dt.predict(df_train[X_all])))`:
    학습 데이터에 대한 예측을 수행하고,
    정확도를 계산하여 `acc_train` 리스트에 추가합니다.
...
```

Image('187.png')

Out[66]: 예시 데이터프레임 `df\_train` 및 `df\_test`

Pclass_1	Pclass_2	Pclass_3	Age	SibSp	Parch	Fare	Embarked_C	Embarked_Q	Embarked_S
0	1	0	22	1	0	7.25	0	0	1
1	0	0	38	1	0	71.28	1	0	0
0	1	0	26	0	0	7.92	0	0	1
...	...	...	...	...	...	...	...	...	...

In [67]: Image('188.png')

Out[67]: `_train` 및 `df_test``

<code>s_3</code>	Age	SibSp	Parch	Fare	Embarked_C	Embarked_Q	Embarked_S	Sex	Survived
	22	1	0	7.25	0	0	1	1	0
	38	1	0	71.28	1	0	0	1	1
	26	0	0	7.92	0	0	1	0	1
	...	...	...	...	...	...	...	...	...

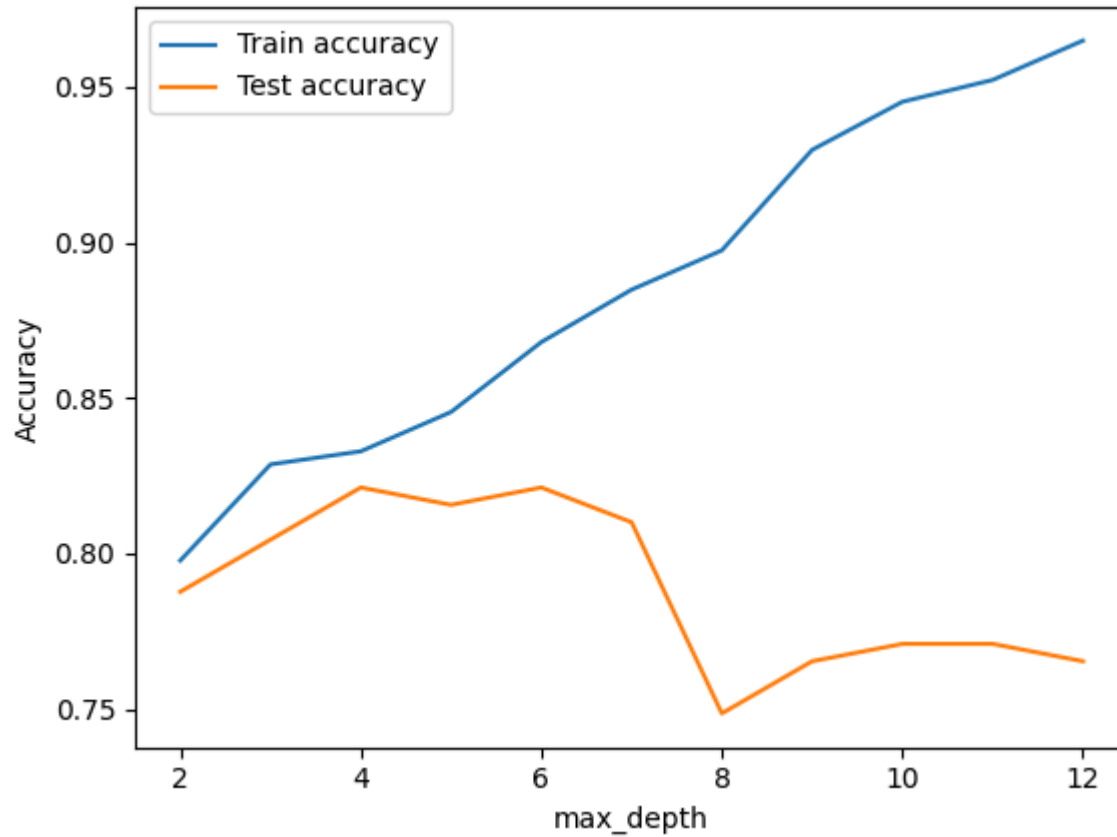
```
In [22]: '''
### 모델 성능 분석
이 코드의 결과로 `acc_train`과 `acc_test`
리스트에는 각 `max_depth` 값에서의 학습 데이터와
테스트 데이터에 대한 정확도가 저장됩니다.
이를 통해 트리의 깊이에 따른 성능 변화를
분석할 수 있습니다.
- 과소적합 (Underfitting):
    트리의 깊이가 너무 작을 때, 모델은 충분히 학습하지
    못하여 학습 데이터와 테스트 데이터 모두에서
    정확도가 낮을 수 있습니다.
- 적절한 학습:
    트리의 깊이가 적절할 때, 모델은 학습 데이터와
    테스트 데이터 모두에서 높은 정확도를 보입니다.
- 과적합 (Overfitting):
    트리의 깊이가 너무 클 때, 모델은 학습 데이터에
    과도하게 맞추어져 테스트 데이터에서의 일반화
    성능이 떨어질 수 있습니다. 이 경우 학습 데이터의
    정확도는 높지만, 테스트 데이터의 정확도는 낮아집니다.

### 최종 목적
이 코드는 다양한 트리 깊이에서 모델의 성능을 평가하여,
최적의 트리 깊이를 찾는 데 도움을 줍니다.
이를 통해 모델이 데이터에 대해 얼마나
잘 일반화할 수 있는지 확인할 수 있습니다.
'''
```

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

max_depths = np.arange(2, 13)
acc_train, acc_test = list(), list()
for max_depth in max_depths:
    clf_dt = DecisionTreeClassifier(max_depth=max_depth,
                                    random_state=123)
    clf_dt.fit(df_train[X_all], df_train['Survived'])
    acc_test.append(
        accuracy_score(df_test['Survived'],
                       clf_dt.predict(df_test[X_all])))
    acc_train.append(
        accuracy_score(df_train['Survived'],
                       clf_dt.predict(df_train[X_all])))
)
```

```
In [23]: plt.plot(max_depths, acc_train,
                  label='Train accuracy')
plt.plot(max_depths, acc_test,
         label='Test accuracy')
plt.gca().set_xlabel('max_depth')
plt.gca().set_ylabel('Accuracy')
plt.legend()
plt.show()
```



max_leaf_nodes 에 따라 학습과 평가의 성능이 어떻게 변하는지 확인합니다.

```
max_leaf_nodes_list = np.arange(2, 10) ** 2
```

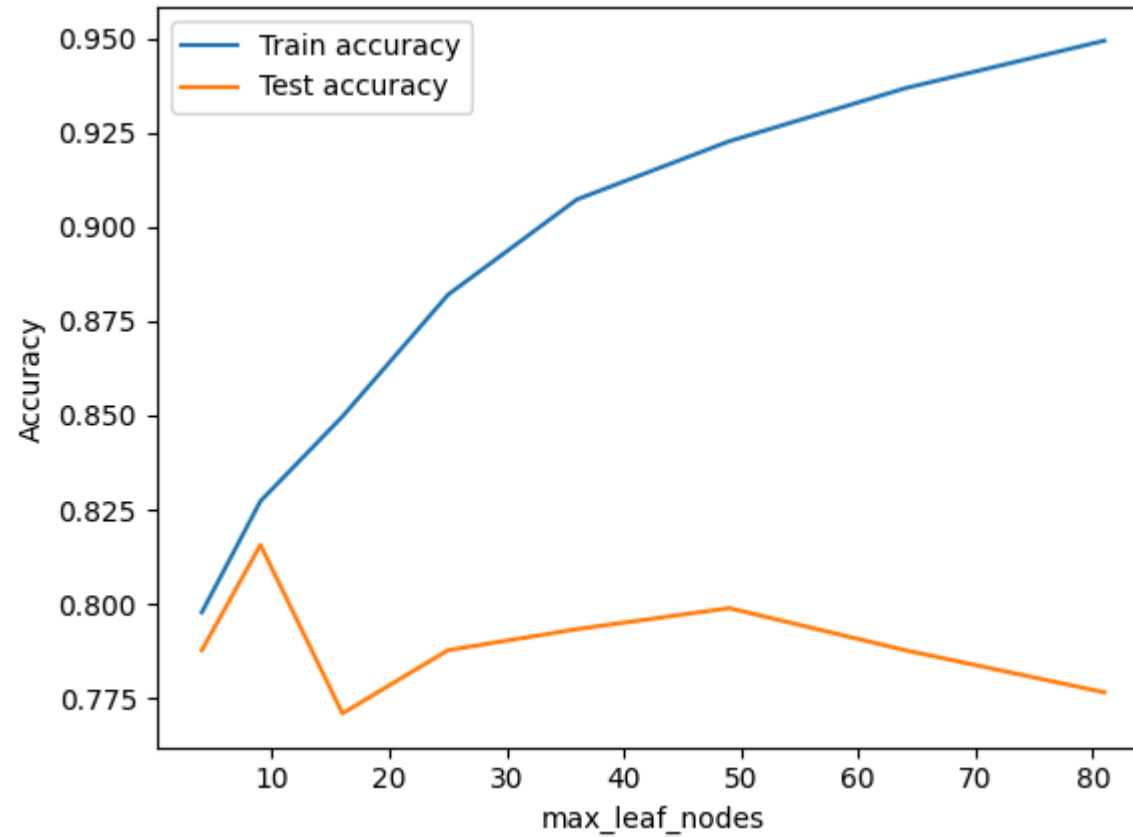
```
In [24]: from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

max_leaf_nodes_list = np.arange(2, 10) ** 2
acc_train, acc_test = list(), list()
for max_leaf_nodes in max_leaf_nodes_list:
    clf_dt = DecisionTreeClassifier(max_leaf_nodes=max_leaf_nodes,
                                    random_state=123)
```



```
clf_dt.fit(df_train[X_all], df_train['Survived'])
acc_test.append(
    accuracy_score(df_test['Survived'],
                   clf_dt.predict(df_test[X_all]))
)
acc_train.append(
    accuracy_score(df_train['Survived'],
                   clf_dt.predict(df_train[X_all]))
)
```

```
In [25]: plt.plot(max_leaf_nodes_list, acc_train,
                  label='Train accuracy')
plt.plot(max_leaf_nodes_list, acc_test,
         label='Test accuracy')
plt.gca().set_xlabel('max_leaf_nodes')
plt.gca().set_ylabel('Accuracy')
plt.legend()
plt.show()
```



min_samples_leaf 에 따라 학습과 평가의 성능이 어떻게 변하는지 확인합니다.

```
min_samples_leafs = np.arange(2, 33, 2)
```

In []:

```
'''
코드는 결정 트리 모델의 최대 잎 노드 수
(max_leaf_nodes)를 다양한 값으로 설정하여 학습시키고,
각 설정에서의 학습 데이터와 테스트 데이터에 대한
정확도를 계산하여 리스트에 저장합니다.
이를 통해 트리의 복잡도에 따른 성능 변화를
분석할 수 있습니다.
'''
```

```

1. `max_leaf_nodes_list` 설정:
- `max_leaf_nodes_list`는 2부터 9까지의 정수의
  제공으로 이루어진 배열입니다. 이는 트리의 최대 잎
  노드 수를 설정할 값들의 범위를 나타냅니다.
- `np.arange(2, 10)`는 2부터 9까지의 정수를 생성하고,
  이를 각각 제공하여 `max_leaf_nodes_list`에 저장합니다.
- 결과적으로 `max_leaf_nodes_list`는
  `[4, 9, 16, 25, 36, 49, 64, 81]`와 같은 값을 가집니다.

2. 정확도 저장을 위한 리스트 초기화:
- `acc_train`과 `acc_test`는 각각
  학습 데이터와 테스트 데이터의 정확도를
  저장할 리스트입니다.

3. 루프를 통한 모델 학습 및 평가:
- `for max_leaf_nodes in max_leaf_nodes_list`:
  `max_leaf_nodes_list` 배열의 각 값에 대해
  반복문을 실행합니다.
- `clf_dt = DecisionTreeClassifier(
  max_leaf_nodes=max_leaf_nodes, random_state=123)` :
  현재 `max_leaf_nodes` 값으로
  결정 트리 모델을 생성합니다.
- `clf_dt.fit(df_train[X_all], df_train['Survived'])` :
  학습 데이터를 사용하여 모델을 학습시킵니다.
- `acc_test.append(accuracy_score(df_test['Survived'],
  clf_dt.predict(df_test[X_all])))` :
  테스트 데이터에 대한 예측을 수행하고,
  정확도를 계산하여 `acc_test` 리스트에 추가합니다.
- `acc_train.append(accuracy_score(df_train['Survived'],
  clf_dt.predict(df_train[X_all])))` :
  학습 데이터에 대한 예측을 수행하고,
  정확도를 계산하여 `acc_train` 리스트에 추가합니다.
...

```

```

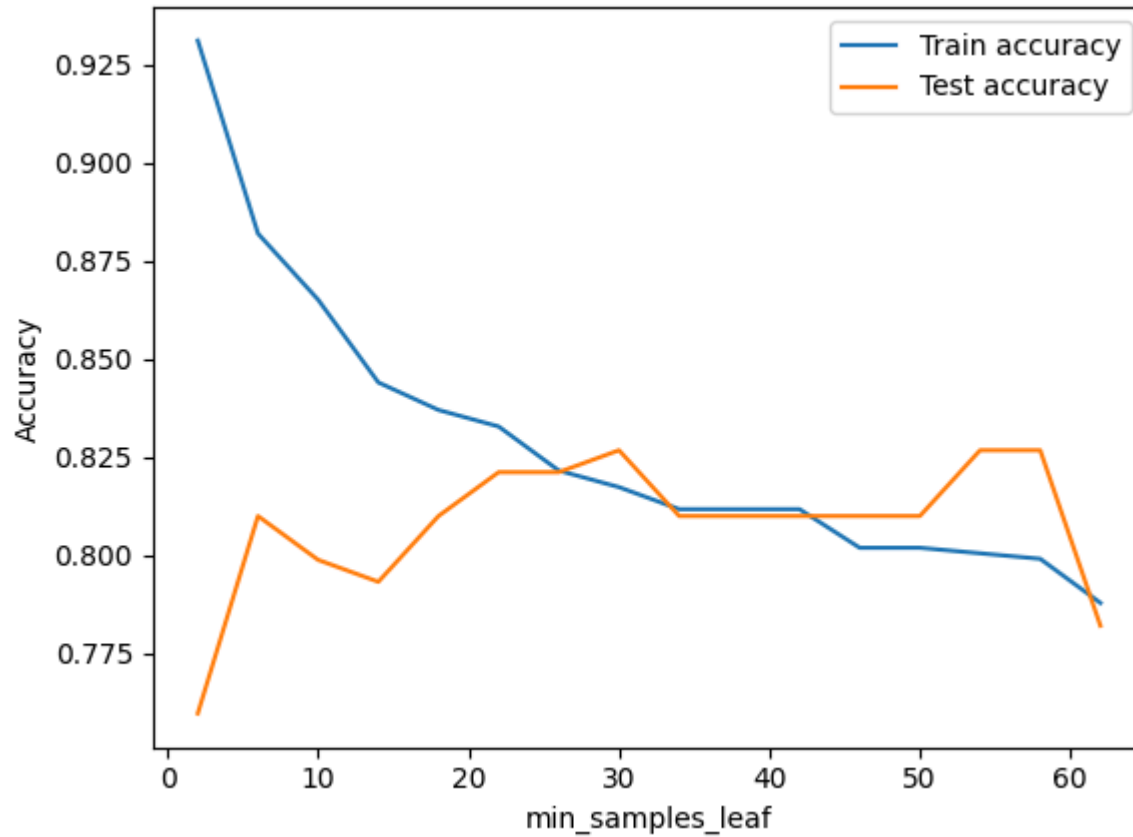
In [26]: from sklearn.tree import DecisionTreeClassifier
         from sklearn.metrics import accuracy_score

         min_samples_leafs = np.arange(2, 64, 4)
         acc_train, acc_test = list(), list()
         for min_samples_leaf in min_samples_leafs :

```

```
clf_dt
    = DecisionTreeClassifier(
        min_samples_leaf=min_samples_leaf,
        random_state=123)
clf_dt.fit(df_train[X_all], df_train['Survived'])
acc_test.append(
    accuracy_score(df_test['Survived'],
        clf_dt.predict(df_test[X_all]))
)
acc_train.append(
    accuracy_score(df_train['Survived'],
        clf_dt.predict(df_train[X_all]))
)
```

```
In [27]: plt.plot(min_samples_leafs, acc_train, label='Train accuracy')
plt.plot(min_samples_leafs, acc_test, label='Test accuracy')
plt.gca().set_xlabel('min_samples_leaf')
plt.gca().set_ylabel('Accuracy')
plt.legend()
plt.show()
```



의사 결정 나무의 장단점

장점

- 데이터의 전처리의 필요성이 적습니다.
- 이상치에 상대적으로 덜 민감합니다.
- 직관적인 해석을 할 수 있는 형태를 보입니다.
- 모델에 가정을 두고 있지 않으므로, 다양한 문제에 적용할 수 있습니다.

단점

- 과적합에 대응하기 위한 모델의 최적화가 상대적으로 많이 필요합니다.
- 데이터의 작은 변화에도 모델의 출력 결과가 크게 변하기 쉽습니다.

In []: